

Nebula: Proving machine executions via folding schemes

Arasu Arun
New York University

Srinath Setty
Microsoft Research

Abstract—A zero-knowledge virtual machine (zkVM) is a versatile, developer-friendly primitive that proves the correct execution of programs on a fixed machine architecture. A major bottleneck for these protocols is the prover’s space complexity, which grows linearly with the number of program steps and makes it impractical to produce proofs for long-running computations. To overcome this, we propose generating these proofs *incrementally*, using folding schemes. However, realizing this requires new tools in the folding setting: (1) an efficient read-write memory argument for proving the correctness of memory operations; and (2) a method to eliminate the overheads incurred by unused machine instructions when incrementally proving a program execution step.

We address these with new techniques. First, we introduce *commitment-carrying IVC*, where a proof carries an incremental commitment to the prover’s non-deterministic advice provided at different steps. Second, we show how this unlocks efficient read-write memory arguments (which implies indexed lookups arguments), with a cost profile identical to that of memory arguments in the context of non-recursive arguments. Third, we provide a new universal “switchboard” circuit construction that combines circuits of different instructions such that one can “turn off” uninvoked circuit elements and constraints, offering a new way to achieve pay-per-use prover costs.

We design an IVC scheme, which we refer to as Nebula, that incorporates these techniques. We implement a prototype of a Nebula-based zkVM for the Ethereum Virtual Machine (EVM). We find that our techniques qualitatively provide a $30\times$ smaller constraint system to represent the EVM over standard memory-checking techniques, and lead to over $260\times$ faster proof generation for the standard ERC-20 token transfer transaction when compared to our baseline Nova (CRYPTO’22) with existing arithmetization methods.

1. Introduction

This paper studies the problem of producing succinct arguments of program executions on machines (e.g., RISC-V, EVM). In this setting, a *prover* and *verifier* are both given the specification of a (virtual) machine (e.g., the instruction set architecture and semantics), which they can preprocess to obtain setup material. After preprocessing, given a program designed to run on such a machine (e.g., specified in its assembly language) along with some inputs, the prover produces a *proof* to convince the verifier that it correctly executed the specified program on the specified inputs to

obtain some claimed outputs. The program can optionally take non-deterministic advice inputs (also called witnesses) from the prover. A proof system satisfies “succinctness” if the length of a proof and the time to verify it are at most polylogarithmic in the time to run the program. Optionally, it may be “zero-knowledge”, meaning that the proof should not reveal anything about the prover’s witnesses beyond what is implied by the statement itself.

These protocols, popularly referred to as “zero-knowledge VMs” (zkVMs),¹ have many applications in decentralized systems. For example, in blockchain rollups, an untrusted off-chain entity uses the proof system to produce a proof of correct execution of arbitrary programs on a specified machine (e.g., EVM). This proof is verified on-chain to ensure that the state change resulting from said programs is reflected correctly on the blockchain [18], [38], [55]. Since verifying a proof is significantly cheaper than executing the original program on-chain, this approach provides scalability and lower transaction costs.

The need for incremental proof generation. A primary bottleneck in realizing an efficient zkVM is the prover’s efficiency, specifically the space efficiency. In state-of-the-art proof systems, producing a proof for an n -step program takes time and space that is at least linear in n , which is prohibitively expensive as the program execution length grows. Many real-world program executions take millions to billions of instructions: hashing 10 KB of data with Keccak256 is about 1 million RISC-V instructions, and checking an Ethereum block can take billions of RISC-V instructions [21].

A natural approach to address space efficiency is *incremental* proof generation: the prover produces a proof for each step (or a batch of sequential steps) of the program execution at once and then updates a “running” proof that proves a prefix of the execution thus far. Incrementally verifiable computation (IVC) [53], [11] formally captures this approach, offering the desirable space efficiency. Thus, we focus on IVC in the rest of the paper.

IVC from folding schemes. A folding scheme [36] is a simple cryptographic primitive in which a prover and a verifier reduce the task of checking two NP instances (e.g., circuit satisfiability instances) into the task of checking a

1. Most zkVMs only offer succinctness, and not the zero-knowledge property. However, most constructions (including the ones in this work) can be made zero-knowledge without significant prover overheads [36], [35].

single instance. They have emerged as the prover-efficient way to realize IVC [36], [33], [35], [17], [27], [34]. In folding-based IVC, the prover maintains a running instance that, along with its associated witnesses, serves as proof of correct execution of the computation thus far. At each IVC step, the prover executes one or more program steps and folds an instance representing the correctness of these steps into the running instance. Folding-based IVC features small recursion overheads, unlike the naive recursion approach that requires the prover to produce a SNARK proof after every IVC step. In folding, the overhead from recursion is the cost of the folding scheme verifier represented as a circuit. For example, in Nova [36], this verifier computes a random linear combination of homomorphic commitments and performs some hashing, costing only 10,000 constraints in R1CS [2]²

Folding-based zkVM.. Designing a folding-based zkVM poses challenges because techniques in monolithic SNARKs [54], [56], [9], [10], [16] cannot be easily translated to the context of IVC. Below, we discuss two challenges.

Challenge 1: Efficient read-write memory. Program executions on machines such as RISC-V or EVM invoke operations that read values from and write values to memory. This requires the prover to prove that these operations were executed correctly (i.e., they are sequentially consistent [37]).

In the IVC setting, a solution to this problem is to maintain a succinct representation of the memory using a vector commitment and verify read and write operations using inclusion proofs. Braun et al. [15] describe a solution that employs Merkle trees as the vector commitment [39], [13]; this approach is also employed in prior zkVMs [11] and even recent folding-based zkVMs [6]. This is, however, very expensive as it involves encoding tens of cryptographic hash operations as constraints. Even with SNARK-friendly hash functions (e.g., Poseidon), where a hash invocation costs about ≈ 250 constraints, each memory operation costs $O(\log M)$ hashes, which is at least 5,000 constraints for a modest-sized 1 MB memory.

A more efficient technique, which is suitable for the IVC setting, is *offline* memory checking [13] using multiset collision-resistant hash functions [23], [44]. Instead of checking each operation individually as in Merkle trees, this approach checks a batch of memory operations at once. The result is that it is more efficient than Merkle trees, both asymptotically and concretely. There is a fixed $O(M)$ cost that amortizes over a batch of memory operations, so on an amortized basis, each memory operation requires only $O(1)$ hash computations. Concretely, this is an order of magnitude more efficient than Merkle trees [15]. Unfortunately, it is orders of magnitude more expensive than memory checking primitives in monolithic proof systems.

In more detail, monolithic proof systems such as Spartan [43], Lasso [49], and Jolt [9] adapt offline memory checking to use fingerprinting-based techniques. Here, the prover commits to

all memory operations (values read and written), the verifier sends a public-coin challenge value (which is chosen via Fiat-Shamir [29] in the non-interactive setting), and the multiset hash function computes a product of values after subtracting the challenge. As a result, proving memory operations (in a monolithic proof system) requires proving only a handful of product operations for each memory operation.

Unfortunately, the fingerprinting-based offline memory checking does not easily carry over to the IVC setting. Specifically, IVC proof generation does not split between a “commit” and a “prove” phase, as witness values are committed incrementally. Furthermore, it is not clear how to ensure that the proofs are incrementally updateable, as the security of the public-coin hash function requires that all witnesses must be known before the challenge is sampled.

A key question is whether one can design a memory checking primitive for IVC that is as efficient as the one in monolithic proof systems.

Challenge 2: “Pay-per-use” costs. Since CPUs and VMs execute programs instruction-by-instruction, the circuit that captures a machine execution of n steps is generally constructed by replicating a per-step circuit capturing a single step of execution. Because the sequence of instructions is not known a priori, the latter must be general enough to capture all possible instructions, leading to a per-step circuit size proportional to the sum of the sizes of the individual instruction circuits. This makes the prover incur enormous per-step proving costs for instructions that are not even executed. Several solutions exist in the literature to address this problem [54], [56], [25], [22], but they focus on monolithic proof systems.

In the IVC context, this is formalized as “non-uniform” IVC [33]: an IVC proof can be incremented using any one among a fixed set of step functions (this set is a singleton in the standard “uniform” IVC) with the prover paying costs proportional to only the executed function (“pay-per-use” cost profile). Note that a generic IVC scheme with a universal circuit that implements a multiplexer over all the supported instructions does *not* satisfy this property: the cost of the prover at each step is proportional to the sum of the circuit sizes of all instructions supported by the machine. SuperNova [33] and Protostar [17] meet the pay-per-use cost requirement but introduce trade-offs. SuperNova can profitably only execute one machine instruction per recursive step, which means it must incur the cost of recursion ($\approx 10,000$ gates) every program step, making it prohibitively expensive for VMs where each instruction performs a small amount of work (e.g., RISC-V). By design, Protostar increases the degree of constraints, which means the approach does not work with R1CS.

A key question is the following: can we get the best of both worlds, where one can execute any number of instructions per IVC step (thereby amortizing the recursion overhead) while paying only for the instructions that were actually

2. For context, a block of SHA-256 costs about 25,000 constraints.

executed (like in SuperNova and Protostar)? Additionally, can we make the approach work with RICS?

1.1. Our contributions

(1) Commitment-carrying IVC. We introduce a generalization of IVC, which we refer to as *commitment-carrying IVC*, that produces a proof along with a commitment to all the non-deterministic advice given by the prover at each step of the IVC. We construct commitment-carrying IVC from folding schemes in a manner analogous to IVC from folding schemes [36]. By taking advantage of commitments to witnesses already computed by the prover in existing folding schemes (e.g., Nova, HyperNova), commitment-carrying IVC incurs minor overheads relative to IVC from folding schemes: the folding verifier performs a few extra hashes per step.

(2) Read-write memory in IVC. We show how the commitment-carrying capability unlocks efficient (offline) memory checking (and thus, lookup arguments) in the IVC setting. In general, the commitment-carrying capability enables the use of randomized checks in IVC’s step computation. The prover can commit to its witness, sample a challenge with Fiat-Shamir, and make the challenge as well as its witness available to the circuit, and the circuit can perform randomized checks (e.g., fingerprinting-based permutation checks). With the commitment-carrying capability, the verifier can check that the challenge was generated after the prover commits to its witness.

(3) Folding IVC proofs to retain incremental proofs. While commitment-carrying IVC allows achieving efficient read-write memory, it alone does not provide incrementality as the memory-checking challenges must still be generated *after* the prover commits to *all* non-deterministic witnesses—and thus, all memory operations—to be used in the proof. This requires us to increment proofs that use different memory-checking challenges. We observe that folding-based IVC proofs are themselves instances of a relation (e.g., committed relaxed RICS in the case of Nova), which means they can be folded together. We leverage this to design a two-layered IVC approach that folds (in the second layer) commitment-carrying IVC proofs of contiguous segments of machine execution (from the first layer), after performing consistency checks (e.g., the starting memory contents of the second IVC proof of machine execution must match the ending memory contents of the first IVC proof of machine execution). This allows us to retain the incrementality features of IVC while leveraging randomized checks for efficient memory.

(4) Switchboard circuits. We describe a new approach to realize non-uniform IVC. This is the first approach that works with RICS while allowing recursion overheads to amortize across a batch of instructions. In a nutshell, we combine a set of circuits into one “switchboard” RICS circuit that can behave as any one of the original circuits, by using “switch” variables that the prover uses to “power down” unused circuits not relevant to the current step. In folding schemes

that only commit to the witness (e.g., HyperNova [35]), this already ensures that the prover’s commitment cost for a VM step depends only on the circuit size associated with the instruction that is executed. This is because committing to zeros is free in MSM-based commitment schemes. We modify Nova [36] to take advantage of this and allow the prover to perform field and group operations proportional only to the instruction executed. Unlike existing constructions, switchboard circuits do not require increasing the degree of the constraints, and thus work with RICS.

Putting it all together: Nebula. The aforementioned techniques together achieve our central result, Nebula, a new non-uniform IVC scheme where the step function gets access to a global memory, while retaining incremental proof generation and the pay-per-use properties. We implement Nebula using Nova’s folding scheme to build a zkVM for a subset of the Ethereum Virtual Machine. Our circuits are small, and the resulting proving costs are two orders of magnitude smaller than the prior best IVC-compatible techniques.

1.2. Related work

We discuss work related to switchboard circuits in Section 5.1, and focus on prior work on memory checking here. Several works provide weaker memory primitives in relaxed variants of IVC. Eagen et al. [28] provide a write-once, read-many memory in RCG, a relaxation of IVC that only supports a bounded number of steps determined a priori. In a similar vein, Mangrove [40] provides a read-only memory in a similarly bounded form of IVC. In contrast with these works, our work provides a general read-write memory primitive with sequential consistency semantics in non-uniform IVC.

In a concurrent and theoretical work, Bunz and Chen [18] provide a read-write memory primitive in IVC. There are several qualitative and quantitative differences between their proposal and Nebula. Appendix B provides details.

Subsequent work. Twist [47] provides a new memory checking argument within a monolithic SNARK. Twist does not efficiently port to the IVC setting: when adapted to the IVC setting, Twist requires the prover to commit to $O(M)$ -sized vectors for *each* IVC step, whereas Nebula incurs such $O(M)$ costs only once when a proof is “finalized”.

2. Preliminaries

We use λ to denote the security parameter and $\mathbb{F}$ to denote a finite field (e.g., the prime field $\mathbb{F}_p$ for a large prime p). We use $\text{negl}(\lambda)$ to denote a negligible function in λ . We write $\Pr[X] \approx \epsilon$ to mean that $|\Pr[X] - \epsilon| = \text{negl}(\lambda)$. Throughout the paper, the depicted asymptotics depend on λ , but we elide this for brevity. We write PPT to refer to probabilistic polynomial time algorithms. We defer a formal definition of commitment schemes to Appendix A.

2.1. Committed relations

Instead of directly working with R1CS or CCS (which generalizes R1CS, Plonkish, and AIR) [48], Nova [36] and HyperNova [35] work with a variant of a relation where a commitment to the witness is in the instance. They generically refer to them as *committed relations*.

Definition 1 (Committed relation). Consider a relation $\mathcal{R}$ over structure, instance, witness tuples where witnesses are in some space W . Consider a commitment scheme $\text{com} = (\text{Gen}, \text{Commit})$ over message space W . We define the corresponding committed relation over public parameter, structure, instance, witness tuples characterized by com as follows. $\mathcal{R}(\text{com}) =$

$$\left\{ (\text{pp}_{\text{com}}, s, (C, u), (w, r)) \mid \begin{array}{l} (s, u, w) \in \mathcal{R}, \\ C = \text{Commit}(\text{pp}_{\text{com}}, w, r) \end{array} \right\}$$

2.2. Folding schemes

A folding scheme [36] is a protocol in which a prover and a verifier reduce the task of checking two instances of a relation $\mathcal{R}$ with some structure s to the task of checking a single instance in $\mathcal{R}$ with the same structure. These protocols are used to construct IVC (and NIVC) schemes. A formal definition of IVC is in Appendix A, and can also be gleaned from the definition of commitment-carrying IVC in Section 3, which generalizes IVC.

Definition 2 (Folding schemes [36]). Let $\mathcal{R}$ be a relation with elements of the form (pp, s, u, w) where pp denotes public parameters, s is some “structure”, u is an instance, and w is a witness. We use the shorthand $(u, w) \in \mathcal{R}$ if pp, s are implied by context. A folding scheme for $\mathcal{R}$ consists of a tuple of PPT algorithms $(\mathcal{G}, \mathcal{K}, \mathcal{P}, \mathcal{V})$.

- $\mathcal{G}(1^\lambda) \rightarrow \text{pp}$ is a generator that takes a security parameter λ and outputs public parameters.
- $\mathcal{K}(\text{pp}, s) \rightarrow (\text{pk}, \text{vk})$ is an encoder that takes a structure s and outputs prover and verifier keys.
- $\mathcal{P}(\text{pk}, (u_1, w_1), (u_2, w_2)) \rightarrow (u, w)$ on input instance-witness tuples (u_1, w_1) and (u_2, w_2) , outputs a new instance-witness tuple (u, w) of the same size.
- $\mathcal{V}(\text{vk}, u_1, u_2) \rightarrow u$ on input instances u_1 and u_2 , outputs a new instance u .

In Nova, during the folding protocol, the prover produces as advice an additional value $\bar{T}$ when folding instances. We use the notation $(u, w, \bar{T}) \leftarrow \text{NIFS}.\mathcal{P}(\text{pk}, (u_1, w_2), (u_1, w_2))$ and $u \leftarrow \text{NIFS}.\mathcal{V}(\text{vk}, u_1, u_2, \bar{T})$ to denote the work done by the Nova prover and verifier on input-witness instances (u_1, w_1) and (u_2, w_2) , respectively. The formal properties satisfied by folding schemes are presented in Appendix A.1.

2.3. Memory checking and multiset hashes

Offline memory checking provides a streaming procedure to check if a sequence of memory operations is consistent

(i.e., a $\text{read}(a)$ returns the value that was last written to address a) [13]. This is in contrast to *online* memory checking protocols (such as the Merkle tree approach discussed in Section 1 earlier), which prove the correctness of each operation separately. Specifically, the offline memory checking protocol of Blum et al. [13] reduces the task of checking memory operations to the task of multiset equality checks and a collection of range checks on timestamp entries in the multisets (Section 4.1 describes the reduction in more detail). When adapted for use in SNARKs (first done in Spice [44]), the multiset equality check is performed on collision-resistant *hashes* of multisets. We refer to Clarke et al. [23] for a formal definition of multiset hashes and present an informal description here: a multiset hash function takes as input multisets of elements from a specific universe such that, for multisets S_1, S_2, S_3 , $\text{hash}(S_1) \cdot \text{hash}(S_2) = \text{hash}(S_3)$ if and only if $S_1 \cup S_2 = S_3$, where $\cdot$ is some operation over the hashes.

There are two known choices for a multiset hash function: (1) a deterministic hash function based on elliptic curves and cryptographic hashing [44]; and (2) a randomized public-coin hash function based on Reed-Solomon fingerprints. Spice instantiated the first option with a traditional hash function composed with a map-to-curve function. It has the advantage that it is straightforward to integrate offline memory checking with IVC: the approach in Spice in the context of SNARKs effectively transfers over to the setting of IVC. The main downside is that such hashes are costly to represent as circuit constraints: in the Spice instantiation, setting Poseidon as the hash function and Elligator-2 [12] as the map-to-curve function, each memory operation costs over 500 constraints.

The second approach computes a randomized “fingerprint” of a multiset by computing a product of the fingerprints of entries in the multiset [43], [49], [9]. The construction works as follows. Let S be a multiset of elements $S = \{e_1, \dots, e_n\}$, where each element is a 3-tuple of elements from a finite field $\mathbb{F}$. That is, $e_i = (a_i, v_i, t_i)$ for $i = 1 \dots n$. Let γ_1, γ_2 be public-coin challenges sampled from $\mathbb{F}$. The public-coin multiset hash, or the fingerprint, of S is $\prod_{i=1}^n (a + \gamma_1 \cdot v + t \cdot \gamma_1^2 - \gamma_2)$. Compared to the first approach, these hashes only require a handful of constraints per entry in the multiset. But the downside is that they are secure only if the public-coin challenges are sampled *after* the prover commits to the entries in the multiset.

As the fingerprint of a multiset is the product of the fingerprints of its elements, SNARKs using this construction end up proving statements about the product of commitment elements. A class of efficient arguments for these types of statements is called “grand product” arguments. The most efficient protocol for this task is that of Thaler [52], which specializes the layered sum-check protocol of Goldwasser, Kalai, and Rothblum (GKR) [30]. To prove that the product of n elements equals a certain value, it requires the prover to perform $O(n)$ field operations and, crucially in practice, does not require the prover to cryptographically commit to any elements larger than the elements of tuples of the multisets.

The principal downside is that the length of the proof is $O(\log^2 n)$. Setty and Lee [46] describe a protocol that makes the prover commit to an additional $O(n)$ -sized vector while providing a proof of length $O(\log n)$. They also describe a hybrid protocol that obtains the same prover time as Thaler, and with a proof length of $O(\log n \log \log n)$.

3. Commitment-carrying IVC

3.1. Formalizing commitment-carrying IVC

For commitment-carrying IVC, we require a commitment scheme (Definition 12) for committing to a sequence of vectors that is also *incremental*: given a commitment C_i for a sequence of vectors $(\omega_0, \dots, \omega_{i-1})$, one can efficiently compute a commitment C_{i+1} for the sequence $(\omega_0, \dots, \omega_{i-1}, \omega_i)$ —while only accessing ω_i and C_i . An example of such a commitment scheme is a hash chain where nodes store vectors and the tail of the hash chain is a commitment to the sequence of vectors: $C_{i+1} \leftarrow H(C_i, \omega_i)$. In Nebula, for efficiency, the incremental commitment scheme (§3.2) hashes *commitments* to witness vectors (e.g., Pedersen [42] or KZG [32]) rather than the vectors themselves. Below, we focus on defining its interface.

Definition 3 (Incremental Commitment Scheme). An incremental commitment scheme is a tuple of PPT algorithms $\text{IC} = (\text{Gen}, \text{Commit}, \text{Open})$.

- $\text{Gen}(\lambda) \rightarrow \text{pp}$: On input security parameter λ , produces public parameters pp .
- $\text{Commit}(\text{pp}, C_i, \omega_i) \rightarrow C_{i+1}$: For some $i \geq 0$, on input a message ω_i , and a prior commitment C_i , with $C_0 = \perp$, outputs an updated commitment C_{i+1} .
- $\text{Open}(\text{pp}, C_{i+1}, [\omega_0, \dots, \omega_i]) \rightarrow \{0, 1\}$: For some $i \geq 0$, on input a commitment C_{i+1} a purported sequence of messages $[\omega_0, \dots, \omega_i]$, outputs a Boolean.

Below, we define commit-carrying IVC, highlighting the difference from the definition of standard IVC in boxes.

Definition 4 (Commitment-carrying IVC). A *commitment-carrying incrementally verifiable computation* (CC-IVC) scheme is defined by PPT algorithms $(\mathcal{G}, \mathcal{P}, \mathcal{V})$ and a deterministic polynomial-time $\mathcal{K}$ denoting the generator, the prover, the verifier, and the encoder respectively, and an incremental commitment scheme IC , with the following interface:

- $\mathcal{G}(1^\lambda, N) \rightarrow \text{pp}$: on input security parameter λ and size bounds N , samples public parameters pp , which includes $\text{pp}_{\text{IC}} \leftarrow \text{IC.Gen}$.
- $\mathcal{K}(\text{pp}, F) \rightarrow (\text{pk}, \text{vk})$: on input public parameters pp , and function F , deterministically produces a prover key pk and a verifier key vk .
- $\mathcal{P}(\text{pk}, (i, z_0, z_i, \text{C}_i), \omega_i, \Pi_i) \rightarrow \Pi_{i+1}$: on input a prover key pk , a counter i , initial input z_0 , initial commitment $C_0 = \perp$, claimed output after i increments

z_i , claimed carried commitment after i increments C_i , a non-deterministic advice ω_i , and an IVC proof Π_i attesting to z_i and C_i , produces a new proof Π_{i+1} attesting to $z_{i+1} = F(z_i, \omega_i)$ and $C_{i+1} = \text{IC.Commit}(\text{pp}_{\text{IC}}, C_i, \omega_i)$.

- $\mathcal{V}(\text{vk}, (i, z_0, z_i, \text{C}_i), \Pi_i) \rightarrow \{0, 1\}$: on input a verifier key vk , a counter i , an initial input z_0 , a claimed output after i increments z_i , a claimed carried commitment after i increments C_i , and an IVC proof Π_i attesting to z_i and C_i , outputs 1 if Π_i is accepting, 0 otherwise.

A CC-IVC scheme $(\mathcal{G}, \mathcal{K}, \mathcal{P}, \mathcal{V})$ satisfies the following.

- 1) **Completeness:** For any PPT adversary $\mathcal{A}$, if:

$$\begin{aligned} & \text{pp} \leftarrow \mathcal{G}(1^\lambda, N) \\ & \wedge (F, (i, z_0, z_i, \text{C}_i), \omega_i, \Pi_i) \leftarrow \mathcal{A}(\text{pp}), \\ & \wedge (\text{pk}, \text{vk}) \leftarrow \mathcal{K}(\text{pp}, F), \\ & \wedge \mathcal{V}(\text{vk}, (i, z_0, z_i, \text{C}_i), \Pi_i) = 1, \\ & \wedge z_{i+1} \leftarrow F(z_i, \omega_i), \\ & \wedge \text{C}_{i+1} \leftarrow \text{IC.Commit}(\text{pp}_{\text{IC}}, C_i, \omega_i), \\ & \wedge \Pi_{i+1} \leftarrow \mathcal{P}(\text{pk}, (i, z_0, z_i, \text{C}_i), \omega_i, \Pi_i), \\ & \wedge b \leftarrow \mathcal{V}(\text{vk}, (i+1, z_0, z_{i+1}, \text{C}_{i+1}), \Pi_{i+1}) \end{aligned}$$

then $\Pr[b = 1] = 1$.

- 2) **Knowledge Soundness:** Consider constant $n \in \mathbb{N}$. For all expected polynomial-time adversaries $\mathcal{P}^*$ there exists an expected polynomial-time extractor $\mathcal{E}$ such that, if,

$$\begin{aligned} & \text{pp} \leftarrow \mathcal{G}(1^\lambda, N) \\ & \wedge (F, (z_0, z, \text{C}), \Pi) \leftarrow \mathcal{P}^*(\text{pp}, r) \\ & \wedge (\text{pk}, \text{vk}) \leftarrow \mathcal{K}(\text{pp}, F) \\ & \wedge (\omega_0, \dots, \omega_{n-1}) \leftarrow \mathcal{E}(\text{pp}, z_0, z, \Pi; r) \\ & \wedge z_{i+1} \leftarrow F(z_i, \omega_i) \quad \forall i \in \{0 \dots n-1\} \\ & \wedge \text{C}_{i+1} \leftarrow \text{IC.Commit}(\text{pp}_{\text{IC}}, C_i, \omega_i) \quad \forall i \in \{0 \dots n-1\} \end{aligned}$$

then $\Pr[z_n \neq z \vee \text{C}_n \neq C] \approx \Pr[\mathcal{V}(\text{vk}, (n, z_0, z, \text{C}), \Pi) = 1]$. where r denotes an arbitrarily long random tape.

- 3) **Succinctness:** The IVC proof size is independent of the iteration count.

3.2. Commitment-carrying IVC from Nova

A straightforward way to construct CC-IVC is to build it atop an existing IVC scheme: augment the step function to maintain and increment the carried commitment. Specifically, consider IVC with a single step function F . The augmented step function F' takes at step i input tuple $z'_i = (z_i, C_i)$ and outputs $(F(z, \omega), \text{IC.Commit}(C_i, \omega_i))$. It is easy to show that any generic IVC scheme instantiated with F' as the step function and initial state $z'_0 = (z_0, \perp)$ realizes commitment-carrying IVC. The drawback, however, is that this transformation requires IC.Commit to be performed as part of the step function, which means capturing it as an

arithmetic circuit. This is prohibitively expensive even with SNARK-friendly primitives as the size of this circuit is linear in the size of the witness.

To avoid this overhead, we leverage the fact that known folding schemes already require the prover to compute commitments to witnesses at each step. For example, Nova and HyperNova, which work with *committed* relations (Definition 1), satisfy this property. We now show how to transform the Nova folding scheme for these types of relations into a commitment-carrying IVC scheme for the same relation. While we focus on Nova, the approach works for other schemes (e.g., HyperNova).

We first design an incremental commitment scheme by using any commitment scheme and any hash function. This will be instantiated with the commitment scheme underlying the committed relations in the folding scheme.

Construction 1. Let hash be a hash function and $\text{Com} = (\text{Gen}, \text{Commit}, \text{Open})$ be a commitment scheme. Let n denote an upper bound on the size of the witness that will be committed. We define an incremental commitment scheme $\text{IC}_{H, \text{Com}}$ as follows:

$\text{IC.Gen}(1^\lambda, n) \rightarrow \text{pp}$: output $\text{Com.Gen}(\lambda, n)$

$\text{IC.Commit}(\text{pp}, C_{i-1}, \omega_{i-1}) \rightarrow C_i$:

- 1) if $i = 1$, output $\perp$;
- 2) else output $H(C_{i-1}, \text{Com.Commit}(\text{pp}, \omega_{i-1}))$

$\text{IC.Open}(\text{pp}, C_i, [\omega_0, \dots, \omega_{i-1}]) \rightarrow \{0, 1\}$

- 1) set $C'_0 \leftarrow \perp$;
- 2) for $j = 1$ to i :
 - 2.1) compute $C'_j \leftarrow \text{Com.Commit}(\text{pp}, C'_{j-1}, \omega_{i-1})$;
 - 2.2) output 1 if $C_i = C'_i$ and 0 otherwise

We now present the construction of CC-IVC based on Nova's construction of IVC from folding schemes [36, Construction 3]. We highlight the changes from the original construction in boxes, and refer the reader to Section 5 of Nova for further background and intuition. For a committed instance u , we use $u.x$ to access its public input, and $u.C_W$ to access its commitment to the witness. In our context, the witness is *split-committed*: the witness at step i is represented as a tuple $(\omega_{i-1}, \text{aux}_{i-1})$, with each component committed separately; this is to separate parts of the witness that are carried in the incremental commitment and parts that are not carried.

Construction 2 (CC-IVC from folding schemes). Consider a committed relation $\mathcal{R}$ with a commitment scheme $\text{Com} = (\text{Gen}, \text{Commit}, \text{Open})$. Let NIFS be an IVC-compatible non-interactive folding scheme for $\mathcal{R}$. Let $(u_\perp, w_\perp)$ be a default instance-witness pair for $\mathcal{R}$ that satisfies any structure and public parameters. Consider a polynomial-time function F that takes a non-deterministic input, and a cryptographic hash function hash . We now

define the augmented function F' , where all input arguments are taken as non-deterministic advice.

$F'(\text{vk}, U_i, u_i, (i, z_0, z_i, \boxed{C_{i-1}}), \omega_i, \bar{T}, r_i, r_{i+1}) \rightarrow x$:
if $i = 0$,

- 1) set $U_{i+1} = u_\perp$ and $\boxed{C_0} = \perp$
- 2) output $\text{hash}(\text{vk}, i+1, z_0, F(z_0, \omega_i), U_{i+1}, \boxed{C_0}, r_{i+1})$.

otherwise,

- 1) compute $z_{i+1} \leftarrow F(z_i, \omega_i)$
- 2) check that $u_i.x = \text{hash}(\text{vk}, i, z_0, z_i, U_i, \boxed{C_{i-1}}, r_i)$
- 3) set $U_{i+1} \leftarrow \text{NIFS.V}(\text{vk}, U_i, u_i, \bar{T})$
- 4) parse $u_i.C_W$ as $(C_{\omega_{i-1}}, C_{\text{aux}_{i-1}})$ and compute $\boxed{C_i} \leftarrow \text{hash}(\text{pp}, \text{pp}_{\text{IC}}, C_{i-1}, C_{\omega_{i-1}})$
- 5) output $x \leftarrow \text{hash}(\text{vk}, i+1, z_0, z_{i+1}, U_{i+1}, \boxed{C_i}, r_{i+1})$.

As F' is a polynomial-time function, it can be encoded as a committed-relaxed RICS structure $s_{F'}$. We let $(u_{i+1}, w_{i+1}) \leftarrow \text{trace}(F', (\text{vk}, U_i, u_i, (i, z_0, z_i), \omega_i, \bar{T}, r_i, r_{i+1}))$ be the function that produces a satisfying instance-witness pair on given inputs for this structure. We now define the CC-IVC scheme $(\mathcal{G}, \mathcal{K}, \mathcal{P}, \mathcal{V})$.

$\mathcal{G}(1^\lambda, N) \rightarrow \text{pp}$: output $(\text{IC.Gen}(1^\lambda, N), \text{NIFS.G}(1^\lambda, N))$.

$\mathcal{K}(\text{pp}, F) \rightarrow (\text{pk}, \text{vk})$:

- 1) encode $s_{F'} \leftarrow \text{enc}_{\text{str}}(F')$
- 2) compute $(\text{pk}_{\text{fs}}, \text{vk}_{\text{fs}}) \leftarrow \text{NIFS.K}(\text{pp}, s_{F'})$
- 3) output $(\text{pk}, \text{vk}) \leftarrow ((\text{pp}, F, \text{pk}_{\text{fs}}), (\text{pp}, s_{F'}, \text{vk}_{\text{fs}}))$

$\mathcal{P}(\text{pk}, (i, z_0, z_i, \boxed{C_i}), \omega_i, \Pi_i) \rightarrow \Pi_{i+1}$:
if $i = 0$,

- 1) let $(U_{i+1}, W_{i+1}, \bar{T}) \leftarrow (u_\perp, w_\perp, \perp)$.

otherwise,

- 1) parse Π_i as $((U_i, W_i), (u_i, w_i), \boxed{C_{i-1}}, r_i)$.
- 2) parse $u_i.C_W$ as $(C_{\omega_{i-1}}, C_{\text{aux}_{i-1}})$ and abort if $\boxed{C_i} \neq \text{hash}(C_{i-1}, C_{\omega_{i-1}})$
- 3) fold $(U_{i+1}, W_{i+1}, \bar{T}) \leftarrow \text{NIFS.P}(\text{pk}_{\text{fs}}, (U_i, W_i), (u_i, w_i))$
- 4) sample $r_{i+1} \in \mathbb{F}$ randomly
- 5) compute $(u_{i+1}, w_{i+1}) \leftarrow \text{trace}(F', (\text{vk}, U_i, u_i, (i, z_0, z_i, \boxed{C_{i-1}}), \omega_i, \bar{T}), r_i, r_{i+1})$
- 6) output $\Pi_{i+1} \leftarrow ((U_{i+1}, W_{i+1}), (u_{i+1}, w_{i+1}), \boxed{C_i}, r_{i+1})$

$\mathcal{V}(\text{vk}, (i, z_0, z_i, \boxed{C_i}), \Pi_i) \rightarrow \{0, 1\}$:

- 1) if $i = 0$, output 1 if $z_i = z_0$ and 0 otherwise.
- 2) parse Π_i as $((U_i, W_i), (u_i, w_i), \boxed{C_{i-1}}, r_i)$, and check $u_i.x = \text{hash}(\text{vk}_{\text{fs}}, i, z_0, z_i, U_i, \boxed{C_{i-1}}, r_i)$
- 3) parse $u_i.C_W$ as $(C_{\omega_{i-1}}, C_{aux_{i-1}})$, and check that $\boxed{C_i = \text{hash}(C_{i-1}, C_{\omega_{i-1}})}$
- 4) check that W_i and w_i are satisfying witnesses to U_i and u_i , respectively.

The proofs of the following theorems follow directly from minor modifications to the proofs provided in Nova [36].

Theorem 5 (CC-IVC from folding schemes). Construction 2 is a CC-IVC scheme.

Proof (Intuition). Given a malicious prover $\mathcal{P}_i$ that produces a convincing CC-NIVC proof (Π_i, C_i) of i iterations with non-negligible probability, we can construct an extractor $\mathcal{E}_{i-1}$ that extracts a proof Π_{i-1} along with witness ω_{i-1} by the knowledge soundness of the underlying folding scheme. Note that Π_i contains C_{i-1} and u_i from which $C_{\omega_{i-1}}$ can be parsed. (Π_i, C_i) being convincing implies that $C_i = \text{hash}(C_{i-1}, C_{\omega_{i-1}})$. As the commitments are binding, $C_{\omega_{i-1}} = \text{Com.Commit}(\text{pp}_{\text{IC}}, \omega_{i-1})$. Thus, $C_i = \text{IC.Commit}(\text{pp}_{\text{IC}}, C_{i-1}, \omega_{i-1})$ as desired. This lets us construct a new prover $\mathcal{P}_{i-1}^*$ that uses $\mathcal{E}_{i-1}$ to output a convincing proof (Π_{i-1}, C_{i-1}) . We can now recursively repeat this process with $\mathcal{P}_{i-1}^*$ to construct an extractor that extracts the witness at each step of CC-NIVC. $\square$

Lemma 6 (Overheads of CC-IVC over IVC). In Construction 2, the overhead to support CC-IVC on top of Nova’s IVC is as follows. The committed instances contain an additional commitment (as the commitment to the witness at each step is split). The size of a CC-IVC proof is larger by two commitments and a hash value. The augmented step function F' now performs an additional hash computation and a group scalar multiplication operation (to increment the carried commitment)

Proof (Intuition). The increase in the sizes of committed instances is due to their holding a commitment to the non-deterministic witness ω provided at each step. The extra work performed by F' is in incrementing the carried commitment at each step i as $C_i \leftarrow \text{hash}(C_{i-1}, C_{\omega_{i-1}})$. Note that this cost is independent of $|\omega_{i-1}|$. Furthermore, since the non-deterministic advice is committed separately, the verifier circuit performs an additional group scalar multiplication. $\square$

4. Efficient read-write memory in IVC

This section describes how to devise an efficient read-write memory primitive in IVC using commitment-carrying IVC.

For ease of exposition, we focus on IVC, but our description generalizes easily to non-uniform IVC (NIVC).

4.1. Problem statement and an overview of Nebula

When the step-function F represents the transition function of a CPU or virtual machine (e.g., RISC-V, EVM), F must be able to read from and write to a *global* memory. By global, we mean that if a particular value is written to memory at step i , a subsequent read operation performed at steps $j \geq i$ must reflect the effects of that write. This is formally captured by sequential consistency [37]. In a bit more detail, suppose that the prover maintains a memory M , where M is a vector of a certain size. During invocations of F , it may perform one or more memory operations, where an operation can be either a read or a write. For $\text{read}(a)$, where a is some address (i.e., index) in M , an honest prover provides the value currently stored at index a in M . For $\text{write}(a, v)$, where a is some address and v is some value, an honest prover stores v at index a in M . We wish to design an IVC scheme where the prover proves the correct execution of invocations of F , including the correctness of all memory operations performed during those invocations. In addition, the prover’s space requirement should be proportional to $O(|M| + |F|)$.

Nebula employs fingerprinting-based offline memory checking [13], [23], [8], [44], [43], [49], [9]. Recall from Section 2.3 that fingerprint-based offline memory checking uses multiset hash functions based on Reed-Solomon fingerprints and requires all elements of the multiset to be committed before the public-coin challenges are sampled. In monolithic proof systems (e.g., Jolt [9]), this requirement is not a problem: the prover executes the program until completion, commits to the required multisets, computes the challenges, and then proves the correctness of memory operations.

This is, however, not straightforward to do in the IVC (or folding) setting. First, the commitment to the prover’s witness is not produced at once and is instead split across multiple IVC steps. Moreover, the verifier does not have access to these commitments (providing these would violate succinctness as the proof size would be proportional to the number of IVC steps). Second, an IVC proof must be incrementally updateable. For example, the prover can pause the execution of a VM, externalize a proof of the execution thus far, and then resume the execution of the VM from the paused state. When the prover resumes a paused VM, it should be able to continue updating the proof it has already produced. Nebula solves these issues with commitment-carrying IVC and a novel two-layered IVC approach, respectively.

4.2. Background: memory checks to multiset checks

We adapt this subsection from prior work [49], [43], [31]. We generalize their description as a reduction from read-write memory checking to grand product checks rather than from read-only memory checking to grand product checks.

Recall that in offline memory checking [13], a *trusted checker* issues operations to an untrusted memory. To enable efficient checking using multiset-fingerprinting techniques, the memory is modified so that in addition to storing a value at each address, the memory also stores a timestamp with each address. Moreover, each read operation is followed by a write operation that updates the timestamp associated with that address (but not the value stored there). Similarly, each write operation is preceded by a read operation that reads the current timestamp associated with that address and writes back a new value and a new timestamp. We depict the memory-checking procedure in the codeboxes below.

Local state of the checker. Three sets: IS, RS, and WS (“initial set”, “read set”, and “write set”, respectively), which are initialized as follows.³ $RS = WS = \{\}$, and for an M -sized memory, IS is initialized to the following set of tuples: for all $i \in [M]$, the tuple (i, v_i, it_i) is included in IS, where v_i is the initial value stored at address i and it_i is the initial timestamp associated with the value (e.g., 0). Here, $[M]$ denotes the set $\{0, 1, \dots, M-1\}$. Additionally, the checker maintains a timestamp counter ts that is initialized to the highest timestamp value in IS.

Memory operations and an invariant. For a read operation at address a , suppose that the untrusted memory responds with a value-timestamp pair (v, t) . Then the checker updates its local state as follows:

- 1) $ts \leftarrow ts + 1$
- 2) assert $t < ts$
- 3) $RS \leftarrow RS \cup \{(a, v, t)\}$;
- 4) store (v, ts) at address a in the untrusted memory;
- 5) $WS \leftarrow WS \cup \{(a, v, ts)\}$.

For a write operation at address a that wishes to write a value of v' , suppose that the untrusted memory responds with a value-timestamp pair (v, t) . Then the checker updates its local state as follows:

- 1) $ts \leftarrow ts + 1$
- 2) assert $t < ts$
- 3) $RS \leftarrow RS \cup \{(a, v, t)\}$;
- 4) store (v', ts) at address a in the untrusted memory;
- 5) $WS \leftarrow WS \cup \{(a, v', ts)\}$.

The following lemma captures the invariant maintained on the sets of the checker is captured by the following lemma, a more general form of which is proven in [45, Lemma C.1].

Lemma 7. Let IS, WS, and RS denote the (initial, write and read) multisets maintained by the checker in the above algorithm at the conclusion of memory operations. If for every read operation, the untrusted memory returns the tuple last written to that location, then there exists a set FS (“final set”) with cardinality M consisting of tuples of the form (i, v'_i, t_i) for all $k \in [M]$ such that $IS \cup WS = RS \cup FS$. Moreover, FS is computable in time linear in M (i.e., FS is

3. The checker in [13] maintains a fingerprint of these sets, but for our exposition, we let the checker maintain full sets.

the final state of memory viewed as a set of address-value-timestamp tuples). Conversely, if the untrusted memory ever returns a value v for a memory call $k \in [M]$ such that v does not equal the value last written to cell k , then there does not exist any set FS such that $IS \cup WS = RS \cup FS$.

Corollary 8. Let $\mathbb{F}$ be a prime order field. Assuming that the domain of timestamps is $\mathbb{F}$ and that the number of memory operations issued N is smaller than the field characteristic $|\mathbb{F}|$. Let $IS = \{(i, v_i, t_i)\}_{i=0}^{M-1}$, $WS = \{(wa_i, wv_i, wt_i)\}_{i=0}^{N-1}$, and $RS = \{(ra_i, rv_i, rt_i)\}_{i=0}^{N-1}$ denote the multisets maintained by the checker in the above algorithm at the conclusion of memory operations. If for every read operation, the untrusted memory returns the tuple last written to that location, then there exists a set FS with cardinality M consisting of tuples of the form (i, v'_i, t'_i) for all $i \in [M]$ such that, the following holds with probability 1 over the choice of $\gamma_1, \gamma_2 \in \mathbb{F}$: letting $f(a, v, t) = a + \gamma_1 \cdot v + \gamma_2^2 \cdot t - \gamma_2$,

$$\prod_{i=0}^{M-1} f_{\gamma_1, \gamma_2}(a, v_i, t_i) \cdot \prod_{i=0}^{N-1} f_{\gamma_1, \gamma_2}(wa_i, wv_i, wt_i) = \prod_{i=0}^{N-1} f_{\gamma_1, \gamma_2}(ra_i, rv_i, rt_i) \cdot \prod_{i=0}^{M-1} f_{\gamma_1, \gamma_2}(i, v'_i, t'_i)$$

Moreover, FS is computable in time linear in M (i.e., FS is the final state of memory viewed as a set of address-value-timestamp tuples).

Conversely, if the untrusted memory ever returns a value v for a memory call $k \in [M]$ such that v does not equal the value last written to cell k , then for any set FS, the above equality checks holds with probability at most $O(M+N)/|\mathbb{F}|$, where the probability is over the choice of $\gamma_1, \gamma_2 \in \mathbb{F}$.

Proof. The desired result follows from using public-coin multiset hash functions ([43, §7.2.1]) in the multiset equality check provided by Lemma 7. $\square$

4.3. Details of Nebula’s approach

As discussed earlier, Nebula employs a two-layered IVC scheme to solve the problem introduced earlier. We discuss each component in Nebula’s solution.

(1) Layer 1: Use commitment-carrying IVC to carry untrusted advice regarding memory operations. In our setting, the step function F is allowed to perform read and write memory operations on a global memory. To enable this, F receives additional advice for each memory operation that it invokes. This advice is in the form of *two* (address, value, timestamp) tuples per memory operation. Specifically, for a read operation, the advice is (a, v, rt) and (a, v, wt) ; F checks that the address a in the advice matches the address it requested and then uses the provided value v (e.g., in the rest of its computation). For a write operation, the advice is (a, v, rt) and (a, v', wt) ; F checks that the address a and value v' match the address and value it wishes to write. Otherwise, F ignores the remaining components in the provided advice.

Finally, instead of using IVC to prove the repeated execution of F , Nebula uses commitment-carrying IVC (§3) to carry an incremental commitment to the untrusted advice provided. Let Π_F denote this commitment-carrying IVC proof for an instance (n, z_0, z_n, C_n) after executing n iterations of F with a carried commitment of C_n . Observe that C_n is an incremental commitment to multisets RS and WS in the description provided earlier (§4.2). The circuit for F may use other witness values beyond those involved in memory operations, but we carry over commitment to parts of the witness related to memory.

(2) Finalize the Layer 1 proof Π_F by computing multiset hashes underneath C_n . The proof Π_F is incrementally updateable (i.e., it is a commitment-carrying IVC proof). The prover can continue to update it by applying additional iterations of F . Thus, a verifier verifying Π_F must also validate that multisets underneath C_n respect sequential consistency. The latter requires the prover to send a preimage of C_n and for the verifier to compute multiset hashes, which we avoid in the following way.

We devise two functions F_{ops} and F_{scan} to compute multiset hashes of (RS, WS) and (IS, FS) respectively. They also perform the necessary timestamp checks on entries in RS and WS (§4.2). We depict these functions in the following codeboxes.

Hash($\gamma_1, \gamma_2, a, v, t$) $\rightarrow h$:

1) return $(a + v \cdot \gamma_1 + t \cdot \gamma_1^2 - \gamma_2)$

$F_{\text{ops}}(z_i = (\gamma_1, \gamma_2, ts, h^{\text{RS}}, h^{\text{WS}}), \omega_i = (\text{RS}, \text{WS})) \rightarrow z_{i+1}$:

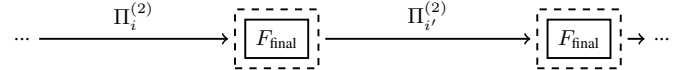
1) assert $|\text{RS}| = |\text{WS}|$
 2) for i in $0..|\text{RS}|$,
 a) $(a, v, rt) \leftarrow \text{RS}[i]$
 b) $(a', v', wt) \leftarrow \text{WS}[i]$
 c) $ts \leftarrow ts + 1$
 d) assert $rt < ts$
 e) assert $wt = ts$
 f) $h^{\text{RS}} \leftarrow h^{\text{RS}} \cdot \text{Hash}(\gamma_1, \gamma_2, a, v, rt)$
 g) $h^{\text{WS}} \leftarrow h^{\text{WS}} \cdot \text{Hash}(\gamma_1, \gamma_2, a', v', wt)$
 3) return $(\gamma_1, \gamma_2, ts, h^{\text{RS}}, h^{\text{WS}})$

$F_{\text{scan}}(z_i = (\gamma_1, \gamma_2, h^{\text{IS}}, h^{\text{FS}}), \omega_i = (\text{IS}, \text{FS})) \rightarrow z_{i+1}$:

1) assert $|\text{IS}| = |\text{FS}|$
 2) for i in $0..|\text{IS}|$,
 a) $(a, v, it) \leftarrow \text{IS}[i]$
 b) $(a', v', ft) \leftarrow \text{FS}[i]$
 c) assert $a = a' = i$
 d) $h^{\text{IS}} \leftarrow h^{\text{IS}} \cdot \text{Hash}(\gamma_1, \gamma_2, a, v, it)$
 e) $h^{\text{FS}} \leftarrow h^{\text{FS}} \cdot \text{Hash}(\gamma_1, \gamma_2, a', v', ft)$
 3) return $(\gamma_1, \gamma_2, h^{\text{IS}}, h^{\text{FS}})$

F_{scan} and F_{ops} require public challenges $\gamma_1, \gamma_2 \in_R \mathbb{F}$ as

Layer 2:



Layer 1:

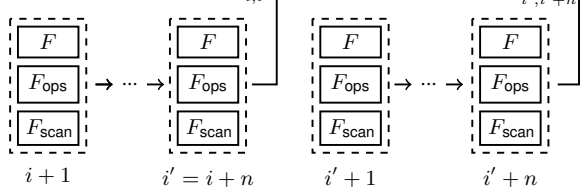


Figure 1: Nebula’s two-layer approach to memory-checking that retains incrementality. The dashed rectangles represent the CC-IVC prover executions of the relevant step functions.

initial input. For the soundness guarantees of Lemma 8, these challenges must be sampled after the prover has committed to its multisets. For this, the prover first computes an incremental commitment to multisets IS and FS (wlog, assume that these multisets, viewed as a vector of tuples, are chunked into n pieces, where n is the number of folding steps to be executed in this proof). Note that this requires a linear scan over the initial and final states of memory. It then derives challenges by hashing those commitments along with C_n . Finally, Nebula uses commitment-carrying IVC to iteratively prove the correct execution of F_{ops} and F_{scan} .

Let Π_{ops} denote the commitment-carrying IVC proof for the correct execution of n invocations of F_{ops} with the instance $(n, (\gamma_1, \gamma_2, ts, h_0^{\text{RS}}, h_0^{\text{WS}}), (\gamma_1, \gamma_2, ts', h_n^{\text{RS}}, h_n^{\text{WS}}), C_n^{\text{ops}} = (C_n^{\text{RS}}, C_n^{\text{WS}}))$. Similarly, let Π_{scan} denote the commitment-carrying IVC proof for the correct execution of n invocations of F_{scan} with the instance $(n, (\gamma_1, \gamma_2, h_0^{\text{IS}}, h_0^{\text{FS}}), (\gamma_1, \gamma_2, h_n^{\text{IS}}, h_n^{\text{FS}}), C_n^{\text{scan}} = (C_n^{\text{IS}}, C_n^{\text{FS}}))$.⁴ The verifier checks the following in addition to verifying $\Pi_F, \Pi_{\text{ops}}, \Pi_{\text{scan}}$:

- 1) check $h_0^{\text{IS}} = h_0^{\text{RS}} = h_0^{\text{WS}} = h_0^{\text{FS}} = 1$
- 2) check $C_n^{\text{ops}} = C_n$
- 3) check γ_1 and γ_2 are derived by hashing C_n^{ops} and C_n^{scan}
- 4) check $h_n^{\text{IS}} \cdot h_n^{\text{WS}} = h_n^{\text{RS}} \cdot h_n^{\text{FS}}$

(3) Layer 2: Fold IVC proofs to restore incrementality. Although Π_F is incrementally updateable to include additional iterations of F beyond n iterations, Π_{ops} and Π_{scan} are *not*. This is because the statement that they prove depends on public-coin challenges obtained using the commitments carried at the end of the n th step. One option is to incrementally update Π_F for some number of steps $n' > n$, and then regenerate Π_{ops} and Π_{scan} using a new $C_{n'}$. Unfortunately, the cost to produce these latter proofs grows linearly with n' rather than $n' - n$.

4. Here, we are carrying commitments to IS and FS separately, to facilitate carrying the commitment to final state as the commitment to the initial state in the next iteration in the second layer of IVC.

Our core observations are as follows. First, we can view $(\Pi_F, \Pi_{\text{ops}}, \Pi_{\text{scan}})$ as a “finalized” IVC proof for the statement $(n, z_0, z_n, C^{\text{FS}})$, where n is the number of iterations of F applied, z_0 is the initial input, z_n is the final output, and C^{FS} is the second commitment carried by Π_{scan} to the final contents of memory. Second, C^{FS} depends only on the final state of memory and not on the challenges γ_1, γ_2 . Third, we can fold the finalized IVC proof into a “running” IVC proof while carrying the statement proven and the final state of memory. Figure 1 visually depicts this, and we now elaborate.

How are IVC proofs themselves folded together? In folding-scheme-based IVC, an IVC proof is a set of instances and their associated witnesses. For example, with Nova [36], an IVC proof is a relaxed RICS instance U and an RICS instance u , and their associated witnesses W and w . So, there is a natural folding scheme for two IVC proofs $\Pi_1 = (U_1, u_1, W_1, w_1)$ and $\Pi_2 = (U_2, u_2, W_2, w_2)$. In particular, the folding scheme for IVC proofs folds the underlying instances into a single relaxed RICS instance and the associated witness. Such a folding of IVC proofs can be realized with Nova (and other IVC schemes) by simply devising a step circuit that implements the IVC folding verifier, and that step circuit is tiny: $\approx 40,000$ gates to fold four instances into one with Nova’s folding scheme [2].

In more detail, we build the following step circuit whose goal is to fold the instances underlying finalized IVC proofs (the running proof of execution, and the current segment), perform some consistency checks, and forward the statement proven by the finalized IVC proof in its public IO. The running state thus far is represented by the tuple $(i, z_0, z_i, ts_i, C_i^{\text{mem}}, U_i)$ and the proof of the latest segment is given by $\Pi_{i,i+n}$. The core consistency check ensures that C_i^{mem} , the (incremental) commitment to the final state of memory after step i , is equal to the commitment to the initial state of memory used in the proof of the latest segment to be folded.

$$F_{\text{final}}((i, z_0, z_i, ts_i, C_i^{\text{mem}}, U_i), \Pi_{i,i+n}) \\ \rightarrow (i+n, z_0, z_{i+n}, ts_{i+n}, C_{i+n}^{\text{mem}}, U_{i+n})$$

- 1) parse $\Pi_{i,i+n}$ to obtain $(U_F, U_{\text{ops}}, U_{\text{scan}})$ and public IO in those IVC proofs $(i+n, z'_i, z_{i+n}, ts'_i, ts_{i+n}, \gamma_1, \gamma_2, C^{\text{IS}}, C^{\text{FS}}, C^{\text{RS}}, C^{\text{WS}}, h^{\text{RS}}, h^{\text{FS}}, h^{\text{WS}}, h^{\text{IS}})$.
- 2) check that $z'_i = z_i$ and $ts'_i = ts_i$
- 3) check that $C_i^{\text{mem}} = C^{\text{IS}}$ // latest program segment initializes memory consistently
- 4) check that γ_1, γ_2 are sampled by hashing $C^{\text{RS}}, C^{\text{WS}}, C^{\text{IS}}, C^{\text{FS}}$.
- 5) check that $h^{\text{RS}} \cdot h^{\text{FS}} = h^{\text{WS}} \cdot h^{\text{IS}}$
- 6) fold instances $(U_F, U_{\text{ops}}, U_{\text{scan}})$ into U_i to get U_{i+n}
- 7) output $(i+n, z_0, z_{i+n}, ts_{i+n}, C_{i+n}^{\text{mem}} = C^{\text{FS}}, U_{i+n})$

Theorem 9. Nebula is an IVC scheme where the step

function F can read from and write to a global memory with sequential consistency semantics. The prover’s space requirements are $O(|F| + |M|)$, where $|M|$ is the size of the global memory.

Proof (sketch). Completeness is easy to check. By invoking the knowledge soundness of the underlying CC-IVC schemes, we can extract witnesses supplied to produce Nebula’s IVC proof, including the multisets whose fingerprints were computed using CC-IVC. By the soundness of Lemma 8, we have that unless the prover respects sequential consistency, the probability that the multiset hash equality checks pass on the extracted vectors is bounded by $O(n + |M|)/|\mathbb{F}|$, which is negligible in the security parameter since $|\mathbb{F}| \approx 2^\lambda$. $\square$

4.4. Prover costs and an optimization.

Figure 2 depicts Nebula’s prover’s costs for proving memory operations over a memory of size $|M|$. The costs involve committing to two (address, value, timestamp) tuples of field elements per memory operation. These field elements are “small”, i.e., they reside in a small subset of $\mathbb{F}$ if memory is, say, byte addressable. Additionally, the prover commits to three arbitrary field elements and proves three constraints for each invocation of Hash in F_{ops} and F_{scan} .⁵ For each folding step, there is overhead in terms of group scalar multiplications and hashing, but this is relatively low.⁶ The cost of proving F_{scan} , which is incurred once when a proof is finalized (in Layer 1), amortizes over all the memory operations performed in Layer 1.

Optimization: computing the multiset hash function outside the circuit. In the solution described thus far, a step circuit computes running products to compute a multiset hash of read sets and write sets. This means that the witness the prover commits to contains intermediate product values involved in the multiset hash calculation. As these values can be very large (up to the size of the field), committing to them is costly. We now sketch a method, called Nebula-Opt, to reduce this commitment work substantially. The core idea is to compute the multiset grand product using the protocol of Setty and Lee [46, §6] *outside* the circuit. The circuit then simply folds the verifier of this protocol using techniques from HyperNova [35]. As a result, the in-circuit work reduces to roughly logarithmic in the number of items multiplied together. Outside the circuit, when using the hybrid protocol of Setty and Lee, the prover only needs to commit to a small fraction of the intermediate product values.

5. NIVC using a universal switchboard circuit

This section describes a new approach for achieving non-uniform IVC (NIVC), where in an IVC scheme one can

5. This can be reduced to one constraint and one arbitrary field element if we employ CCS [48] to devise a custom constraint system for computing the hash function.

6. As memory checking witnesses corresponding to each of the four sets are committed to separately, the costs from Lemma 6 are quadrupled here.

Method	Committed elems per op.	Constraint counts	
		per op.	per folding step
Nebula	6 small, 6 arb	6	$4 H + 4 G $
Nebula-Opt	6 small	6	$4 H + 4 G + O(\log M)$

Figure 2: Nebula’s prover’s costs in the memory checking primitive for a memory of size $|M|$. We capture these costs in terms of the number of witness elements committed and the R1CS constraints involved. We separate the constraints into those that are incurred for every memory operation and once for an entire IVC step. “Small” and “arb” (for arbitrary) $\mathbb{F}$ refer to the cost of committing to field elements from a bounded set (say from $\{0, 1, \dots, \log |M|\}$) and of field elements from the entire field. $|H|$ is the cost of hashing commitments arising from CC-IVC and can be efficient when using a circuit-friendly hash function (e.g., Poseidon). $|G|$ refers to the cost of a group scalar multiplication. For brevity, we omit the costs of performing a scan over the entire memory. Such a scan is performed only once when finalizing proofs (in Layer 1) and are amortized over all the memory operations proven by the IVC proof. Because of this amortization, the costs arising from this scan at most double the memory checking costs as the number of memory operations is at least the number of cells in memory.

execute any number of functions per step (like in Nova [36]) while paying only for the functions that were actually executed (like in SuperNova [33]). Furthermore, this approach works with R1CS without increasing the degree of constraints.

Our core idea is to build a universal switchboard circuit that combines multiple circuits into a single circuit such that the prover can “turn on” one of them and the prover’s cost is proportional only to the size of the active circuit.

Note that for commonly used functionalities (e.g., hashing, digital signature verification), custom circuits are much more efficient than proving those functionalities via the primitive instruction set of a machine (e.g., with bitwise operations, integer arithmetic). Non-uniform IVC allows invoking custom circuits as necessary without paying for them when uninvoked.

5.1. An overview of our approach

Let ℓ denote the number of functions supported by non-uniform IVC. Suppose that $C_1, \dots, C_\ell$ denote their R1CS representations. We construct a universal “switchboard” circuit, specified by another R1CS instance C^* , with “switch” variables that let the prover “turn on” one of $\{C_1, \dots, C_\ell\}$ while “powering down” the others. One can then use β copies of C to execute β functions per recursive step of a *uniform* IVC scheme (e.g., Nova [36]). Since there is a single circuit to execute β functions, this solution only requires a single invocation of the folding scheme for β functions, which amortizes recursion overheads over β functions. The

Technique	Degree Increase	Added Constraints	Active Variables
Multiplexer	0	$\ell(x + 1)$	$O(n \cdot \ell)$
Ginger [50]	1	$\ell + 1$	$O(n)$
Geppetto [24]	1	$\ell + 1$	$O(n)$
Protostar [17]	1	$\ell + 1$	$O(n)$
Switchboard	0	$\ell(x + 1) + 1$	$O(n)$

Figure 3: Comparing the costs of universal circuit constructions for combining ℓ different circuits of m constraints and n variables each. The multiplexer method, which is not “pay-per-use”, executes all ℓ circuits and uses additional binary flags to select the relevant output. Techniques that increase the degree by 1 can be made to work with R1CS constraints but with the cost of doubling the total number of constraints because one must rewrite constraints to lower the degree of constraints. Active variables refer to variables with non-zero values that count towards the prover’s commitment costs.

latter property is unlike SuperNova [33], where each function requires an invocation of the folding scheme verifier.

Prior work. A simple way to “power down” a set of constraints is to multiply constraints by a new variable $s \in \{0, 1\}$. Suppose that a constraint checks $x = y$, the transformed constraint is $s \cdot x = s \cdot y$. Observe that the original constraint is enforced when $s = 1$ and is ignored when $s = 0$, allowing the prover to set x and y to 0. This idea appears in Ginger’s arithmetization of conditional control flow [51], Geppetto’s “energy saving circuits” [24], and Protostar’s circuit selection [17]. The issue is that this approach increases the degree of all constraints by 1, which adds to the prover’s work. Furthermore, this approach is incompatible with R1CS and Nova. The original degree could be recovered by splitting the constraints, , but this doubles the number of constraints and the prover’s work. We provide a solution that achieves pay-per-use costs in R1CS without doubling the number of constraints (Figure 3 provides details).

There are two questions to answer: (1) How do we design switch variables to “turn off” certain constraints without increasing the degree of constraints? (2) How do we make sure the prover’s work in the folding scheme is proportional only to the size of the “active” circuit rather than the size of the universal circuit? The rest of this section answers these questions. Although we focus on R1CS for simplicity and efficiency, our approach applies to CCS [48], a generalization of R1CS, Plonkish, and AIR.

5.2. Details of Nebula’s switchboard circuit

Recall that in R1CS, a circuit C is represented with three matrices (A, B, C) . Given an input x and a purported witness w , the circuit is satisfying if $Az \circ Bz = Cz$, where $z = (1, x, w)$. When setting variables of an “inactive” subcircuit to 0, some constraints may still be unsatisfied because of the constant 1 in z . Nebula simply repurposes this and replaces it with a *switch* variable that the prover

can non-deterministically set to either 1 or 0: the variables vector $[1, x, w]$ is now $[s, x, w]$ where s is a new Boolean variable. Now, if we want to turn off a subcircuit, we can set $s = 0$ and then safely set $x = w = 0$. Nebula introduces some constraints to ensure that the switch variables are set correctly. Beyond this, Nebula does not alter the original constraints nor their degree.

Construction 3 (Switchboard Circuit). Let $C_i = (A_i, B_i, C_i)$ with dimensions $m_i \times n_i$ for $i \in \{1, \dots, \ell\}$ denote RICS matrices of the ℓ different functions. Nebula derives RICS matrices of the universal circuit $C^* = (A^*, B^*, C^*)$. For ease of exposition, we assume that the ℓ RICS circuits have the same public input and output lengths, but possibly different matrix dimensions. We now describe how the matrices of C^* are constructed and the structure of its witness vector.

Step 1. Stitching submatrices. Let $|x|$ be the size of the public input. Matrices A^*, B^* , and C^* have $m = \sum_{i=1}^{\ell} m_i + 1 + \ell \cdot (1 + |x|)$ rows and $n = 1 + |x| + \sum_{i=1}^{\ell} n_i$ columns. We depict how A^* is constructed pictorially in Figure 4 (B^* and C^* are constructed in a similar manner).

Step 2. Switch constraints. Let z denote the witness vector of the universal circuit, with length $1 + |x| + \sum_{i=1}^{\ell} n_i$. Let $s_1, \dots, s_\ell$ be the indices of the first columns of each matrix: $s_i = 1 + |x| + \sum_{j=1}^{i-1} n_j$ for $i = 1, \dots, \ell$. We introduce additional constraints, and each constraint can be realized with a single row to each of A^*, B^*, C^* .

- 1) *Single switch constraint:* Enforce $\sum_{i=1}^{\ell} z[s_i] = 1$.
- 2) *Binary switch constraints:* For $i \in [1, \ell]$, enforce $z[s_i] \cdot (1 - z[s_i]) = 0$.
- 3) *Input consistency constraints:* For $i \in [1, \ell], j \in [1, |x|]$, enforce $z[j] = z[s_i] \cdot z[s_i + j]$

As a simple example, if circuit 2 is active at a given step with input x and witness w , then the variables are set as follows. First, $s_2 = 1$ and $s_i = 0$ for all $i \neq 2$. The witness vector z for the switchboard will be of the form $[1, x, 0, s_2, x, w, 0 \dots 0]$. Observe that rows corresponding to matrices A_i (similarly B_i and C_i) for $i \neq 2$ result in 0 when multiplied with z so inactive constraints end up checking trivially satisfying constraints $0 \cdot 0 = 0$. We prove the following lemma in Appendix C.1.

Lemma 10. Let C^* be the switch-board circuit generated from circuits $C_1, \dots, C_\ell$. For some input x , if C^* is satisfied by a witness ω^* then we can parse a witness ω from ω^* such that, for some $k \in [\ell]$, subcircuit C_k is satisfied by ω for input x . Moreover, the variables in ω^* associated with turned-off subcircuits C_i for all $i \neq k$ are zeros.

5.3. Making the prover pay for the active circuit

Lemma 10 shows that witnesses associated with inactive constraints can be safely set to zeros. This already allows the prover to not pay for any cryptographic costs in some folding-based IVC schemes (e.g., HyperNova [35], Mova [26]),

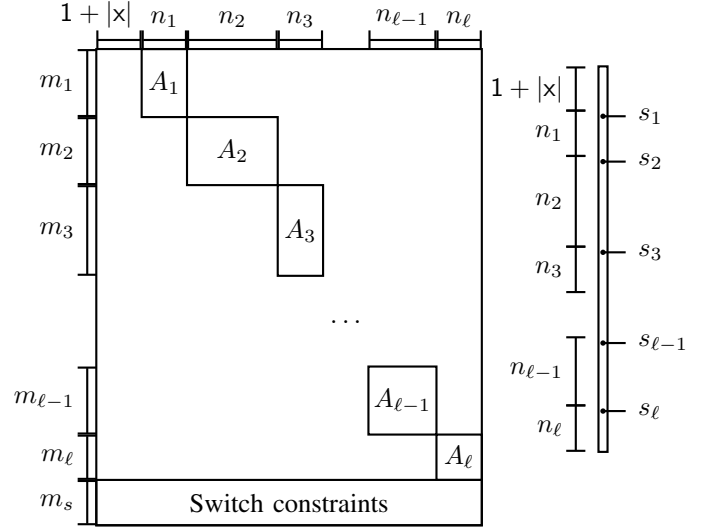


Figure 4: A depiction of Construction 3, to stitch ℓ RICS matrices into a single matrix. On the left is matrix A (matrices B, C are constructed similarly) and the right is vector z . Here, $m_s = 1 + \ell(1 + |x|)$.

where the prover only commits to a witness and committing to zeros is free. Whereas, some folding schemes (e.g., Nova [36], Ova [4]) require the prover to commit to a cross-term, which can be a vector of arbitrary field elements even if the witness contains zeros. We now modify Nova such that its cross-term commitment cost can be made proportional to the size of the active circuit rather than the size of the total switch-board circuit, which leads to a non-uniform IVC scheme. We provide a sketch highlighting the modifications, and provide a full proof in Appendix C.2.

Theorem 11. Let $F_1, \dots, F_\ell$ and φ be the step functions and selector function in non-uniform IVC, respectively. Let $C_1, \dots, C_\ell$ be the RICS circuits representing each of those ℓ functions (let these circuits internally embed the computation of φ and check if it selects the circuit it is embedded in). Let each C_i have dimensions $m_i \times n_i$ for $i \in [1, \ell]$. Let C^* denote the switchboard circuit constructed with $(C_1, \dots, C_\ell)$. By using C^* with Nova’s uniform IVC scheme, we obtain a non-uniform IVC scheme that can execute any number of functions per recursive step.

Proof (sketch). We need to show that C^* with Nova satisfies the “pay-per-use” efficiency criteria of non-uniform IVC. Recall that Nova folds two relaxed RICS instances satisfying the following equations: $Az_1 \circ Bz_1 = u_1 Cz_1 + E_1$, and $Az_2 \circ Bz_2 = u_2 Cz_2 + E_2$. Recall that in Nova’s setting of IVC, $E_2 = 0$ and $u_2 = 1$, so the cross-term $T \in \mathbb{F}^m$ that the prover commits is given by:

$$T = Az_1 \circ Bz_2 + Az_2 \circ Bz_1 - u_1 Cz_2 - Cz_1$$

We can rewrite the computation of $\bar{T}$ as follows:

$$\bar{T} = (\overline{Az_1 \circ Bz_2}) + (\overline{Az_2 \circ Bz_1}) - u_1 \cdot \overline{Cz_2} - \overline{Cz_1}$$

In our context, if z_2 is m -sparse (i.e., there are m non-zero entries and $m = n_{active} + 1 + \ell \cdot (1 + |x|)$), Az_2 , Bz_2 , and Cz_2 vectors will also be m -sparse. This is because in the switchboard circuit construction, each subcircuit only accesses witnesses associated with the subcircuit. The only exception is constraints associated with switch variables. However, there are at most $1 + \ell \cdot (1 + |x|)$ of these constraints, so the sparsity of Az_2, Bz_2, Cz_2 is still guaranteed. Thus, the prover can compute $\overline{Az_1 \circ Bz_2}$ and $\overline{Az_2 \circ Bz_1}$ and $u_1 \overline{Cz_2}$ with $O(m)$ work.

Finally, we need to show that $\overline{Cz_1}$ can be computed with $O(m)$ work. The key idea here is to have the prover cache $\overline{Cz_1}$ and efficiently update it in each iteration of IVC: update $z_1 \leftarrow z_1 + r \cdot z_2$, so the prover can do the following $\overline{Cz_1} \leftarrow \overline{Cz_1} + r \cdot \overline{Cz_2}$. At initialization, $\overline{Cz_1}$ is a commitment to the zero vector, and the prover knows $\overline{Cz_2}$, so the prover can update $\overline{Cz_1}$ with constant work. $\square$

6. Implementation and evaluation

We implement Nebula’s techniques, memory checking and switchboard circuits, atop the public Nova implementation [2], and use it to build a subset of the Ethereum virtual machine (EVM) [1]. In this section, we briefly describe the Nebula-based EVM implementation and measure the impact of Nebula’s techniques on proving the validity of Ethereum transactions.

6.1. EVM memory and operations

Bodies of memory. There are five bodies of memory in an EVM program: the program code, the stack, the RAM, the calldata, and the storage. Each step of the EVM reads the current operation from the program code. Then, most instructions pop operands from the stack, perform the operation, and push results back into the stack. Instructions may load values from and store values to the RAM, which is volatile and cleared after each program. Storage is a long-term, key-value store with 256-bit keys and values, and is persistent across programs. Memory and storage accesses are restricted to a few dedicated instructions each.

Sizes. Program code, RAM, and calldata can be represented as standard byte-addressable arrays. While calldata and RAM do not have a fixed upper bound on size, using them costs “gas” (the EVM’s measure of computation work done by an operation), which limits the total work done by an Ethereum block. For the ERC-20 program, we assume that the calldata and memory can be up to 2^{16} in length, which is a reasonable estimate given the nature of the program and the gas costs involved. The stack can reach a maximum height of 1024 elements, each of 256-bit length. As these values are bigger

Method	Memory Body / Total Size			
	Code/ 2^{16}	Stack/ 2^{10}	RAM/ 2^{16}	Storage/ 2^{20}
Merkle Tree	3750	5000	8000	-
Spice*	1100	1100	1100	1100
Nebula	4	4	4	4
+Range Checks	rc_m	rc_m	rc_m	rc_m
+Scans	$2^{19}/m$	$2^{13}/m$	$2^{19}/m$	$2^{23}/m$

Figure 5: Microbenchmarks comparing different memory-checking techniques applied to different memories in the EVM. The cost depicted is that of a write, except for program code which is read-only. m denotes the number of memory operations performed. The sizes of program code, stack and RAM are $2^{16}, 2^{10}, 2^{16}$, respectively. For storage, we assume that about 2^{20} keys are initialized. We denote the cost of a range check involving values of up to m by rc_m . Each of the two scans performed in in Nebula for a body of memory M costs under $4 \cdot |M|$ constraints. (*For Spice, range check and scan costs are ignored for brevity.)

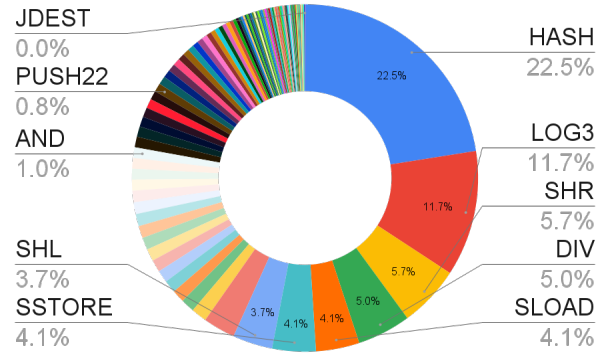


Figure 6: A depiction of the sizes of the 100 opcode circuits in our prototype EVM implementation.

than the field elements used in our implementation, they are represented with two field elements, slightly increasing the cost of the multiset hashes performed for memory-checking (§4.3).

Figure 5 shows the amortized number of constraints (i.e., including the linear scans of memory performed in Spice and Nebula) required to encode a write operation (except for program code, which is read-only). For the sizes of the data structures seen in the EVM, Nebula’s cost per operation is much lower than that obtained by using Merkle trees or Spice when the number of operations exceeds 2^{15} per data structure. Most large EVM programs meet this requirement, as the bytecode is read from at least once every program step. However, for storage, the scan cost depends on the total number of keys that are initialized for the contract. As this is persistent, this could be anywhere from a few thousand to tens of millions.

6.2. EVM subcircuits and the ERC-20 transaction

Our proto-EVM implementation supports 100 out of 141 EVM opcodes. Each opcode is represented by a circuit, and the full EVM circuit is a switchboard of these individual opcode circuits. Figure 6 shows the range of subcircuit sizes involved in the 100 EVM opcodes we implemented. For ease of running experiments, we employ a cheaper custom hash function instead of using SHA3 as it is more efficient to prove without affecting the relative comparisons between Nebula and baselines. The total size of the circuit together is about 120,000 gates, and the largest subcircuit (the custom hash) is $\approx 20\%$ of the total size. Note that functionality common to all opcodes, such as the reading of the step’s instruction, is duplicated across all subcircuits. However, due to the switchboard construction, the prover pays only for parts of the circuit that are active at each step.

As shown in Figure 7, we find a $30\times$ reduction in the EVM circuit size when using Nebula’s memory-checking techniques over Spice. Note that this includes constraints for a linear scan over all memory types.

The ERC-20 transaction. The EVM program that we prove using Nebula is the standard ERC-20 token transfer. According to an analysis done in 2020, over 70% of Ethereum transactions are ERC-20 token transfers, and they account for over a third of the gas costs on the blockchain [7]. We obtain the bytecode and calldata for the ERC-20 using the Remix IDE.⁷ The transfer function itself is implemented through the standardized OpenZeppelin [3] library. We use the Istanbul fork of Ethereum for our tests and measurements. Each ERC-20 program takes 635 EVM steps. The costliest operations involved are 3 calls to the hash function, 2 calls to the shift opcodes, 19 memory accesses, and 8 storage accesses.

6.3. Experimental evaluation

Baselines. We build Nebula on Nova as it remains the most optimized publicly available folding scheme implementation.⁸ Since we build on Nova, we use it as a baseline; this also allows us to isolate performance improvements from Nebula’s techniques. We expect a similar performance boost when implementing Nebula’s techniques on top of other folding schemes.

The baseline folding-based zkVM protocol we use is Nova [2] using a universal multiplexer circuit and Spice memory checking [44]. As discussed in Section 4, Spice instantiated with the elliptic-curve based multiset hash function (i.e., without a public-coin challenge) is the most efficient memory checking protocol in the IVC setting, and is significantly more efficient than Merkle trees for the sizes of memories involved in our EVM programs.

⁷. remix.ethereum.org

⁸. There are implementations of other folding schemes such as HyperNova [35] by Sonobe [5] and Nexus [6]. However, they are unoptimized.

Methods			Proving time	
Memory	Switchboard?	Constraints	per txn	Relative
Spice	No	3640 K	OOM	OOM
Spice	✓	3640 K	2400 s	480×
Nebula	No	115 K	1300 s	260×
Nebula	✓	116 K	5 s	1×

Figure 7: Circuit sizes and proving times obtained when proving the correctness of ERC-20 transactions (involving 635 steps) with a combination of Nebula’s techniques and existing baselines. Note that the implementations batch multiple EVM steps per IVC step and the numbers shown are for the best-performing batch size in each setting.

We instantiate Spice’s multiset CRHF, which is a traditional hash function composed with a map-to-curve function, as follows: we use Poseidon for the hash function and Elligator-2 [12] for the map-to-curve function. Assuming that an address-value-timestamp variable can be packed inside two field elements, Spice costs a little over 500 R1CS constraints per multiset hash. Spice computes two multiset hashes and a timestamp range check per memory operation. We perform range checks with simple bit decomposition, which costs $1 + \log m$ constraints, where m is the number of memory operations in the program. (We note that there are more efficient methods, such as lookup arguments, to do this.)

We run our experiments on a commodity laptop with a 2.4GHz 8-core Intel Core i9 processor. Figure 7 depicts our results.

Our benchmarks show a $260\times$ improvement from using a switchboard circuit instead of a multiplexer-based universal circuit. This roughly correlates with the sizes of the operations involved relative to the total circuit size. Most of the commonly used opcodes (such as `PUSH`, `POP`, `DUP`, and their variants) are under 250 constraints, which is around $500\times$ smaller than the full circuit. About a third of the opcodes are between 250 and 1,000 constraints (about $100\times$ smaller than the size of the full circuit). The most expensive operations involved in an ERC-20 transfer are three hash calls ($5\times$ smaller, with our custom hash implementation) and left shift ($25\times$ smaller). When moving from Spice’s memory checking to Nebula’s, we observe a $36\times$ reduction in the circuit size and an outsized $480\times$ improvement in proving time. We believe that this might be due to memory pressure in the baseline. For instance, we find that the baseline’s prover runs out of memory when batching more than one EVM step per Nova step, whereas Nebula’s prover can batch 20 EVM steps per Nova step.

Acknowledgments

We thank Jingchang Sun and Prashanth Ramakrishna for comments on a prior version of this paper.

References

- [1] ETHEREUM: A secure decentralised generalised transaction ledger. <https://ethereum.github.io/yellowpaper/paper.pdf>
- [2] Nova: Recursive SNARKs without trusted setup. <https://github.com/Microsoft/Nova>
- [3] OpenZeppelin: Contracts for secure blockchain applications. <https://docs.openzeppelin.com/contracts/2.x/erc20>
- [4] Ova: A slightly better Nova. <https://hackmd.io/V4838nnlRKal9ZiTHiGYzw>
- [5] sonobe. <https://github.com/privacy-scaling-explorations/sonobe/>
- [6] The Nexus zkVM. <https://github.com/nexus-xyz/nexus-zkvm> (2024)
- [7] Alchemy: Ethereum statistics and data — alchemy (2023), <https://www.alchemy.com/overviews/ethereum-statistics>, accessed: 2023-10-02
- [8] Arasu, A., Eguro, K., Kaushik, R., Kossmann, D., Meng, P., Pandey, V., Ramamurthy, R.: Concerto: A high concurrency key-value store with integrity. In: Proceedings of the ACM International Conference on Management of Data (SIGMOD) (2017)
- [9] Arun, A., Setty, S., Thaler, J.: Jolt: SNARKs for virtual machines via lookups. In: Proceedings of the International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT) (2024)
- [10] Ben-Sasson, E., Chiesa, A., Genkin, D., Tromer, E., Virza, M.: SNARKs for C: Verifying program executions succinctly and in zero knowledge. In: Proceedings of the International Cryptology Conference (CRYPTO) (Aug 2013)
- [11] Ben-Sasson, E., Chiesa, A., Tromer, E., Virza, M.: Scalable zero knowledge via cycles of elliptic curves. In: Proceedings of the International Cryptology Conference (CRYPTO) (2014)
- [12] Bernstein, D.J., Hamburg, M., Krasnova, A., Lange, T.: Elligator: Elliptic-curve points indistinguishable from uniform random strings. In: Proceedings of the ACM Conference on Computer and Communications Security (CCS) (2013)
- [13] Blum, M., Evans, W., Gemmell, P., Kannan, S., Naor, M.: Checking the correctness of memories. In: Proceedings of the IEEE Symposium on Foundations of Computer Science (FOCS) (1991)
- [14] Boneh, D., Chen, B.: LatticeFold+: Faster, simpler, shorter lattice-based folding for succinct proof systems. Cryptology ePrint Archive, Paper 2025/247 (2025), <https://eprint.iacr.org/2025/247>
- [15] Braun, B., Feldman, A.J., Ren, Z., Setty, S., Blumberg, A.J., Walfish, M.: Verifying computations with state. In: Proceedings of the ACM Symposium on Operating Systems Principles (SOSP) (2013)
- [16] Bruestle, J., Gafni, P., the RISC Zero Team: RISC Zero zkVM: Scalable, transparent arguments of RISC-V integrity (2023)
- [17] Bünz, B., Chen, B.: Protostar: Generic efficient accumulation/folding for special sound protocols. In: Proceedings of the International Conference on the Theory and Application of Cryptology and Information Security (ASIACRYPT) (2023)
- [18] Buterin, V.: The dawn of hybrid layer 2 protocols. https://vitalik.ca/general/2019/08/28/hybrid_layer_2.html (Aug 2019)
- [19] Bünz, B., Chen, J.: Proofs for deep thought: Accumulation for large memories and deterministic computations. Cryptology ePrint Archive, Report 2024/325 (2024)
- [20] Bünz, B., Mishra, P., Nguyen, W., Wang, W.: Accumulation without homomorphism. Cryptology ePrint Archive, Paper 2024/474 (2024), <https://eprint.iacr.org/2024/474>
- [21] Carstens, T., Graf, V., Khalil, R., Li, S., Thompson, P., Welz, W.: Announcing zeth: The first type zero zkvm. <https://risczero.com/blog/zeth-release> (2023), accessed: 2025-02-11
- [22] Choudhuri, A.R., Garg, S., Goel, A., Sekar, S., Sinha, R.: Sublonk: Sublinear prover plonk. Proc. Priv. Enhancing Technol. **2024**, 314–335 (2024), <https://api.semanticscholar.org/CorpusID:259324548>
- [23] Clarke, D., Devadas, S., Dijk, M.V., Gassend, B., Edward, G., Mit, S.: Incremental multiset hash functions and their application to memory integrity checking. In: Proceedings of the International Conference on the Theory and Application of Cryptology and Information Security (ASIACRYPT) (2003)
- [24] Costello, C., Fournet, C., Howell, J., Kohlweiss, M., Kreuter, B., Naehrig, M., Parno, B., Zahur, S.: Geppetto: Versatile verifiable computation. In: Proceedings of the IEEE Symposium on Security and Privacy (S&P) (May 2015)
- [25] Di, Z., Xia, L., Nguyen, W., Tyagi, N.: Muxproofs: Succinct arguments for machine computation from vector lookups. In: Proceedings of the International Conference on the Theory and Application of Cryptology and Information Security (ASIACRYPT). p. 236–265 (2024)
- [26] Dimitriou, N., Garreta, A., Manzur, I., Vlasov, I.: Mova: Nova folding without committing to error terms. Cryptology ePrint Archive, Paper 2024/1220 (2024)
- [27] Eagen, L., Gabizon, A.: ProtoGalaxy: Efficient ProtoStar-style folding of multiple instances. Cryptology ePrint Archive, Paper 2023/1106 (2023)
- [28] Eagen, L., Gabizon, A., Sefranek, M., Towa, P., Williamson, Z.J.: Stackproofs: Private proofs of stack and contract execution using Protogalaxy. Cryptology ePrint Archive, Paper 2024/1281 (2024)
- [29] Fiat, A., Shamir, A.: How to prove yourself: Practical solutions to identification and signature problems. In: Proceedings of the International Cryptology Conference (CRYPTO). pp. 186–194 (1986)
- [30] Goldwasser, S., Kalai, Y.T., Rothblum, G.N.: Delegating computation: Interactive proofs for muggles. In: Proceedings of the ACM Symposium on Theory of Computing (STOC) (2008)
- [31] Golovnev, A., Lee, J., Setty, S., Thaler, J., Wahby, R.: Brakedown: Linear-time and field-agnostic SNARKs for RICS. In: Proceedings of the International Cryptology Conference (CRYPTO) (2023)
- [32] Kate, A., Zaverucha, G.M., Goldberg, I.: Constant-size commitments to polynomials and their applications. In: Proceedings of the International Conference on the Theory and Application of Cryptology and Information Security (ASIACRYPT). pp. 177–194 (2010)
- [33] Kothapalli, A., Setty, S.: SuperNova: Proving universal machine executions without universal circuits. Cryptology ePrint Archive, Report 2022/1758 (2022)
- [34] Kothapalli, A., Setty, S.: CycleFold: Folding-scheme-based recursive arguments over a cycle of elliptic curves. Cryptology ePrint Archive, Report 2023/1192 (2023)
- [35] Kothapalli, A., Setty, S.: HyperNova: Recursive arguments for customizable constraint systems (2024)

- [36] Kothapalli, A., Setty, S., Tzialla, I.: Nova: Recursive Zero-Knowledge Arguments from Folding Schemes. In: Proceedings of the International Cryptology Conference (CRYPTO) (2022)
- [37] Lamport, L.: How to make a multiprocessor computer that correctly executes multiprocess programs. IEEE Transactions on Computers **C-28**(9) (Sep 1979)
- [38] Lee, J., Nikitin, K., Setty, S.: Replicated state machines without replicated execution. In: Proceedings of the IEEE Symposium on Security and Privacy (S&P) (2020)
- [39] Merkle, R.C.: A digital signature based on a conventional encryption function. In: Proceedings of the International Cryptology Conference (CRYPTO) (1988)
- [40] Nguyen, W., Datta, T., Chen, B., Tyagi, N., Boneh, D.: Mangrove: A scalable framework for folding-based snarks. Cryptology ePrint Archive, Report 2024/416 (2024)
- [41] Nguyen, W., Setty, S.: Neo: Lattice-based folding scheme for CCS over small fields and pay-per-bit commitments. Cryptology ePrint Archive, Paper 2025/294 (2025), <https://eprint.iacr.org/2025/294>
- [42] Pedersen, T.P.: Non-interactive and information-theoretic secure verifiable secret sharing. In: Proceedings of the International Cryptology Conference (CRYPTO) (1991)
- [43] Setty, S.: Spartan: Efficient and general-purpose zkSNARKs without trusted setup. In: Proceedings of the International Cryptology Conference (CRYPTO) (2020)
- [44] Setty, S., Angel, S., Gupta, T., Lee, J.: Proving the correct execution of concurrent services in zero-knowledge. In: Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI) (Oct 2018)
- [45] Setty, S., Angel, S., Gupta, T., Lee, J.: Proving the correct execution of concurrent services in zero-knowledge (extended version). ePrint Report 2018/907 (Sep 2018)
- [46] Setty, S., Lee, J.: Quarks: Quadruple-efficient transparent zkSNARKs. Cryptology ePrint Archive, Report 2020/1275 (2020)
- [47] Setty, S., Thaler, J.: Twist and shout: Faster memory checking arguments via one-hot addressing and increments. Cryptology ePrint Archive, Paper 2025/105 (2025), <https://eprint.iacr.org/2025/105>
- [48] Setty, S., Thaler, J., Wahby, R.: Customizable constraint systems for succinct arguments. Cryptology ePrint Archive (2023)
- [49] Setty, S., Thaler, J., Wahby, R.S.: Unlocking the lookup singularity with Lasso. In: Proceedings of the International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT) (2024)
- [50] Setty, S., Vu, V., Panpalia, N., Braun, B., Ali, M., Blumberg, A.J., Walfish, M.: Taking proof-based verified computation a few steps closer to practicality (extended version). ePrint Report 2012/598 (2012)
- [51] Setty, S., Vu, V., Panpalia, N., Braun, B., Blumberg, A.J., Walfish, M.: Taking proof-based verified computation a few steps closer to practicality. In: Proceedings of the USENIX Security Symposium (Aug 2012)
- [52] Thaler, J.: Time-optimal interactive proofs for circuit evaluation. In: Proceedings of the International Cryptology Conference (CRYPTO) (2013)
- [53] Valiant, P.: Incrementally verifiable computation or proofs of knowledge imply time/space efficiency. In: Theory of Cryptography Conference (TCC). pp. 552–576 (2008)
- [54] Wahby, R.S., Setty, S., Ren, Z., Blumberg, A.J., Walfish, M.: Efficient RAM and control flow in verifiable outsourced computation. In: Proceedings of the Network and Distributed System Security Symposium (NDSS) (2015)
- [55] WhiteHat, B., Gluchowski, A., HarryR, Fu, Y., Castonguay, P.: Roll_up / roll_back snark side chain ~17000 tps. <https://ethresear.ch/t/roll-up-roll-back-snark-side-chain-17000-tps/3675> (Oct 2018)
- [56] Zhang, Y., Genkin, D., Katz, J., Papadopoulos, D., Papamanthou, C.: vRAM: Faster verifiable RAM with program-independent preprocessing. In: Proceedings of the IEEE Symposium on Security and Privacy (S&P) (2018)

Appendix A. Deferred Definitions

Definition 12 (Commitment Scheme). A commitment scheme is defined by polynomial-time algorithm $\text{Gen} : \mathbb{N}^2 \rightarrow P$ that produces public parameters given the security parameter and size parameter, a deterministic polynomial-time algorithm $\text{Commit} : P \times M \times R \rightarrow C$ that produces a commitment in C given a public parameters, message, and randomness tuple such that binding holds. That is, for any PPT adversary $\mathcal{A}$, given $\text{pp} \leftarrow \text{Gen}(\lambda, n)$, and given $((m_1, r_1), (m_2, r_2)) \leftarrow \mathcal{A}(\text{pp})$ we have that

$$\Pr[(m_1, r_1) \neq (m_2, r_2) \wedge \text{Commit}(\text{pp}, m_1, r_1) = \text{Commit}(\text{pp}, m_2, r_2)] \approx 0.$$

The commitment scheme is deterministic if Commit does not use its randomness.

Definition 13 (Hiding). The commitment scheme $(\text{Gen}, \text{Commit})$ is hiding if for any PPT adversary $\mathcal{A}$, given $\text{pp} \leftarrow \text{Gen}(\lambda, n)$, $((m_1, r_1), (m_2, r_2)) \leftarrow \mathcal{A}(\text{pp})$, and $C_i \leftarrow \text{Commit}(\text{pp}, m_i, r_i)$ for $i \in \{1, 2\}$ we have that

$$\Pr[\mathcal{A}(\text{pp}, C_1) = 1] \approx \Pr[\mathcal{A}(\text{pp}, C_2) = 1].$$

Definition 14 (Homomorphic). The commitment scheme $(\text{Gen}, \text{Commit})$ is homomorphic if the message space M , randomness space R , and commitment space C are groups and for all $n \in \mathbb{N}$, and $\text{pp} \leftarrow \text{Gen}(\lambda, n)$, we have that for any $m_1, m_2 \in M$ and $r_1, r_2 \in R$

$$\begin{aligned} \text{Commit}(\text{pp}, m_1, r_1) + \text{Commit}(\text{pp}, m_2, r_2) \\ = \text{Commit}(\text{pp}, m_1 + m_2, r_1 + r_2). \end{aligned}$$

Definition 15 (Succinct Commitments). A commitment scheme $(\text{Gen}, \text{Commit})$, over message space M and commitment space R , provides succinct commitments if for all $\text{pp} \leftarrow \text{Gen}(1^\lambda)$, and any $m \in M$ and $r \in R$, we have that $|\text{Commit}(\text{pp}, m, r)| = O_\lambda(\text{polylog}(|m|))$.

Definition 16 (Multilinear Polynomial Commitment Scheme). A multilinear polynomial commitment scheme over polynomial ring $\mathbb{F}^1[X_1, \dots, X_n]$ is a commitment scheme $(\text{Gen}, \text{Commit})$ over message space $\mathbb{F}^1[X_1, \dots, X_n]$, equipped with an argument of knowledge (Definition 17)

for relation $\mathcal{R}_{\text{polyeval}}$ defined as a tuple $(\text{pp}, (C, x, y), (P, r))$ such that:

$$\left\{ (\text{pp}, (C, x, y), (P, r)) \mid \begin{array}{l} P \in \mathbb{F}^1[X_1, \dots, X_n], \\ P(x) = y, \\ C = \text{Commit}(\text{pp}, P, r) \end{array} \right\}.$$

Definition 17 (Argument of Knowledge). Consider relation $\mathcal{R}$ over public parameters, structure, instance, and witness tuples. A reduction of knowledge for $\mathcal{R}$ is defined by PPT algorithms $(\mathcal{G}, \mathcal{P}, \mathcal{V})$ and deterministic algorithm $\mathcal{K}$, denoting the generator, the prover, the verifier and the encoder respectively with the following interface.

- $\mathcal{G}(\lambda, N) \rightarrow \text{pp}$: Takes as input security parameter λ and size parameters N . Outputs public parameters pp .
- $\mathcal{K}(\text{pp}, s) \rightarrow (\text{pk}, \text{vk})$: Takes as input public parameters pp and structure s . Outputs prover key pk and verifier key vk .
- $\mathcal{P}(\text{pk}, u, w) \rightarrow \perp$: Takes as input public parameters pp , and an instance-witness pair (u, w) . Interactively proves that $(\text{pp}, s, u, w) \in \mathcal{R}$.
- $\mathcal{V}(\text{pk}, u) \rightarrow \{0, 1\}$: Takes as input public parameters pp , and an instance u . Interactively checks u .

Let $\langle \mathcal{P}, \mathcal{V} \rangle$ denote the interaction between $\mathcal{P}$ and $\mathcal{V}$. We treat $\langle \mathcal{P}, \mathcal{V} \rangle$ as a function that takes as input $((\text{pk}, \text{vk}), u, w)$ and runs the interaction on prover input (pk, u, w) and verifier input (pp, u) . At the end of the interaction, $\langle \mathcal{P}, \mathcal{V} \rangle$ outputs the verifier's decision. An argument of knowledge $(\mathcal{G}, \mathcal{K}, \mathcal{P}, \mathcal{V})$ satisfies the following conditions.

- 1) **Completeness:** For any PPT adversary $\mathcal{A}$, given $\text{pp} \leftarrow \mathcal{G}(\lambda, N)$, $(s, u, w) \leftarrow \mathcal{A}(\text{pp})$ such that $(\text{pp}, s, u, w) \in \mathcal{R}$ and $(\text{pk}, \text{vk}) \leftarrow \mathcal{K}(\text{pp}, s)$ we have that

$$\langle \mathcal{P}, \mathcal{V} \rangle((\text{pk}, \text{vk}), u, w) = 1$$

- 2) **Knowledge Soundness:** For any expected polynomial-time adversaries $\mathcal{A}$ and $\mathcal{P}^*$, there exists an expected polynomial-time extractor $\mathcal{E}$ such that given $\text{pp} \leftarrow \mathcal{G}(\lambda, N)$, $(s, u, \text{st}) \leftarrow \mathcal{A}(\text{pp})$, and $(\text{pk}, \text{vk}) \leftarrow \mathcal{K}(\text{pp}, s)$, we have that

$$\begin{aligned} & \Pr[(\text{pp}, s, u, \mathcal{E}(\text{pp}, u, \text{st})) \in \mathcal{R}_1] \\ & \approx \Pr[\langle \mathcal{P}^*, \mathcal{V} \rangle((\text{pk}, \text{vk}), u, \text{st}) = 1]. \end{aligned}$$

Definition 18 (Succinctness). An argument of knowledge is succinct if the communication complexity and the verifier time complexity is at most poly-logarithmic in the size of the structure and witness.

Definition 19 (Non-Interactivity). An argument of knowledge is non-interactive if the interaction consists of a single message from the prover to the verifier. In this case, we denote this single message as the output of the prover, and as an input to the verifier.

Definition 20 (Zero-knowledge). An argument of knowledge $(\mathcal{G}, \mathcal{K}, \mathcal{P}, \mathcal{V})$ for relation $\mathcal{R}$ satisfies *zero-knowledge* if for any PPT adversary $\mathcal{V}^*$ there exists an EPT simulator

$\mathcal{S}$ such that for any PPT adversary $\mathcal{A}$ for $\text{pp} \leftarrow \mathcal{G}(1^\lambda, N)$, $(s, (u, w), \text{st}_1) \leftarrow \mathcal{A}(\text{pp})$ such that $(\text{pp}, s, u, w) \in \mathcal{R}$, and $(\text{pk}, \text{vk}) \leftarrow \mathcal{K}(\text{pp}, s)$

$$\{ \text{st}_2 \mid \text{st}_2 \leftarrow \langle \mathcal{P}, \mathcal{V}^*(\text{st}_1) \rangle((\text{pk}, \text{vk}), u, w) \} \cong$$

$$\{ \text{st}_2 \mid \text{st}_2 \leftarrow \mathcal{S}(\text{pp}, s, u, \text{st}_1) \}$$

where st_2 denotes the output of $\mathcal{V}^*$ after interaction. An argument of knowledge satisfies honest-verifier zero-knowledge (HVZK) if it satisfies zero-knowledge under an honest (but curious) verifier that behaves according to the interactive protocol but produces arbitrary output on the side.

Definition 21 (Incrementally Verifiable Computation). An *incrementally verifiable computation* (IVC) scheme is defined by PPT algorithms $(\mathcal{G}, \mathcal{P}, \mathcal{V})$ and a deterministic polynomial-time $\mathcal{K}$ denoting the generator, the prover, the verifier, and the encoder respectively, with the following interface:

- $\mathcal{G}(1^\lambda, N) \rightarrow \text{pp}$: on input security parameter λ and size bounds N , samples public parameters pp .
- $\mathcal{K}(\text{pp}, F) \rightarrow (\text{pk}, \text{vk})$: on input public parameters pp , and function F , deterministically produces a prover key pk and a verifier key vk .
- $\mathcal{P}(\text{pk}, (i, z_0, z_i, \omega_i, \Pi_i) \rightarrow \Pi_{i+1}$: on input a prover key pk , a counter i , initial input z_0 , claimed output after i increments z_i , a non-deterministic advice ω_i , and an IVC proof Π_i attesting to z_i , produces a new proof Π_{i+1} attesting to $z_{i+1} = F(z_i, \omega_i)$.
- $\mathcal{V}(\text{vk}, (i, z_0, z_i, \Pi_i) \rightarrow \{0, 1\}$: on input a verifier key vk , a counter i , an initial input z_0 , a claimed output after i increments z_i , and an IVC proof Π_i attesting to z_i , outputs 1 if Π_i is accepting, 0 otherwise.

An IVC scheme $(\mathcal{G}, \mathcal{K}, \mathcal{P}, \mathcal{V})$ satisfies following requirements.

- 1) **Completeness:** For any PPT adversary $\mathcal{A}$, if:

$$\begin{aligned} & \text{pp} \leftarrow \mathcal{G}(1^\lambda, N) \\ & \wedge (F, (i, z_0, z_i), \omega_i, \Pi_i) \leftarrow \mathcal{A}(\text{pp}), \\ & \wedge (\text{pk}, \text{vk}) \leftarrow \mathcal{K}(\text{pp}, F), \\ & \wedge \mathcal{V}(\text{vk}, (i, z_0, z_i), \Pi_i) = 1, \\ & \wedge z_{i+1} \leftarrow F(z_i, \omega_i), \\ & \wedge \Pi_{i+1} \leftarrow \mathcal{P}(\text{pk}, (i, z_0, z_i, \omega_i, \Pi_i), \\ & \wedge b \leftarrow \mathcal{V}(\text{vk}, (i+1, z_0, z_{i+1}, \Pi_{i+1})) \end{aligned}$$

then $\Pr[b = 1] = 1$.

- 2) **Knowledge Soundness:** Consider constant $n \in \mathbb{N}$. For all expected polynomial-time adversaries $\mathcal{P}^*$ there exists an expected polynomial-time extractor $\mathcal{E}$ such that, if,

$$\begin{aligned} & \text{pp} \leftarrow \mathcal{G}(1^\lambda, N) \\ & \wedge (F, (z_0, z, \Pi) \leftarrow \mathcal{P}^*(\text{pp}, r) \\ & \wedge (\text{pk}, \text{vk}) \leftarrow \mathcal{K}(\text{pp}, F) \\ & \wedge (\omega_0, \dots, \omega_{n-1}) \leftarrow \mathcal{E}(\text{pp}, z_0, z, \Pi; r) \\ & \wedge z_{i+1} \leftarrow F(z_i, \omega_i) \forall i \in \{0 \dots n-1\} \end{aligned}$$

then $\Pr[z_n \neq z] \approx \Pr[\mathcal{V}(\text{vk}, (n, z_0, z, \Pi)) = 1]$. where r denotes an arbitrarily long random tape.

- 3) **Succinctness:** The IVC proof size is independent of the iteration count.

A.1. Properties of folding schemes

Definition 22 (Folding schemes [36]). Let $\mathcal{R}$ be a relation with elements of the form (pp, s, u, w) where pp denotes public parameters, s is some “structure”, u is an instance and w is a witness. We use the shorthand $(u, w) \in \mathcal{R}$ if pp, s are implied by context. A folding scheme for $\mathcal{R}$ consists of a tuple of PPT algorithms $(\mathcal{G}, \mathcal{K}, \mathcal{P}, \mathcal{V})$.

- $\mathcal{G}(1^\lambda) \rightarrow \text{pp}$ is a generator that takes a security parameter λ and outputs public parameters.
- $\mathcal{K}(\text{pp}, s) \rightarrow (\text{pk}, \text{vk})$ is an encoder that takes a structure s and outputs prover and verifier keys.
- $\mathcal{P}(\text{pk}, (u_1, w_1), (u_2, w_2)) \rightarrow (u, w)$ on input instance-witness tuples (u_1, w_1) and (u_2, w_2) , outputs a new instance-witness tuple (u, w) of the same size.
- $\mathcal{V}(\text{vk}, u_1, u_2) \rightarrow u$ on input instances u_1 and u_2 , outputs a new instance u .

We use the notation

$$(u, w) \leftarrow \langle \mathcal{P}(\text{pk}, w_1, w_2), \mathcal{V}(\text{vk}) \rangle(u_1, u_2)$$

to denote an interaction between prover and verifier on comon input (u_1, u_2) and their respective private inputs.

A folding scheme satisfies the following properties:

- *Perfect completeness:* for all satisfying instance-witness pairs (u_1, w_1) and (u_2, w_2) , $(u, w) \leftarrow \langle \mathcal{P}(\text{pk}, w_1, w_2), \mathcal{V}(\text{vk}) \rangle(u_1, u_2)$ is such that (u, w) is a satisfying instance-witness pair.

Formally, if

$$\begin{aligned} \text{pp} &\leftarrow \mathcal{G}(1^\lambda) \\ \wedge (s, (u_1, w_1), (u_2, w_2)) &\leftarrow \mathcal{A}(\text{pp}) \\ \wedge (\text{pp}, s, u_i, w_i) &\in \mathcal{R} \text{ for } i \in \{1, 2\} \\ \wedge (\text{pk}, \text{vk}) &\leftarrow \mathcal{K}(\text{pp}, s) \\ \wedge (u, w) &\leftarrow \langle \mathcal{P}(\text{pk}, w_1, w_2), \mathcal{V}(\text{vk}) \rangle(u_1, u_2) \end{aligned}$$

then $(\text{pp}, s, u, w) \in \mathcal{R}$

- *Knowledge soundness:* for all adversaries $\mathcal{P}^*$, there exists an expected PPT extractor $\mathcal{E}$ such that

$$\begin{aligned} \Pr \left[\begin{array}{l} (\text{pp}, s, u_1, w_1) \in \mathcal{R} \\ (\text{pp}, s, u_2, w_2) \in \mathcal{R} \end{array} \middle| \begin{array}{l} \text{pp} \leftarrow \mathcal{G}(1^\lambda) \\ (s, (u_1, u_2)) \leftarrow \mathcal{P}^*(\text{pp}, \rho) \\ (w_1, w_2) \leftarrow \mathcal{E}(\text{pp}, \rho) \end{array} \right] &\approx \\ \Pr \left[\begin{array}{l} (\text{pp}, s, u_1, w_1) \in \mathcal{R} \\ (\text{pp}, s, u_2, w_2) \in \mathcal{R} \end{array} \middle| \begin{array}{l} \text{pp} \leftarrow \mathcal{G}(1^\lambda) \\ (s, (u_1, u_2)) \leftarrow \mathcal{P}^*(\text{pp}, \rho) \\ (\text{pk}, \text{vk}) \leftarrow \mathcal{K}(\text{pp}, s) \\ (u, w) \leftarrow \langle \mathcal{P}^*(\text{pk}, \rho), \mathcal{V}(\text{vk}) \rangle(u_1, u_2) \end{array} \right] & \end{aligned}$$

Appendix B.

Other related and concurrent work.

In a theoretical work, Bunz and Chen [19] provide a read-write memory primitive in IVC. There are several qualitative and quantitative differences between their proposal and Nebula. First, their approach does not work with R1CS as their folding verifier circuit is of high degree (4–7 depending on the variant). Second, their approach is tied to additively homomorphic commitments, so it is not compatible with folding schemes based on non-homomorphic commitments [20] or lattice commitments [41], [14]. Third, their base protocol requires committing to higher number of arbitrary-length field elements (9 as shown in [19, Table 1] vs 6 in Nebula) per memory access, which significantly increases the concrete prover commitment cost. Their optimized version avoids this, but instead it requires the verifier circuit to encode $(c+1) \cdot \log M$ group scalar multiplications, where M is the size of memory, and c is an arbitrary constant. Even for tiny memories, this requires hundreds of scalar multiplications (several hundreds of thousands of constraints) in the verifier circuit. In contrast, an optimized version of Nebula (§4), which also avoids committing to arbitrary field elements, only requires $O(\log M)$ hashes, which is significantly cheaper than their solution. Fourth, the prover must prove a statement proportional to M . Whereas, in our construction, since the memory itself is chunked into smaller segments, the prover can use an IVC scheme to prove the statement over the final state of the memory. This approach is fundamentally incompatible with theirs because their mem-update protocol requires homomorphic commitments to memory contents whereas our approach does not. Finally, their construction requires multiple sub-protocols: permutation argument, memory-update protocol, logupGKR, bivariate sum-check, etc. making the overall construction far more complex than ours.

Appendix C.

Deferred Proofs

C.1. Proof of Lemma 10

Proof. Let $z = (1, x, w^*)$ denote the variables vector satisfying circuit C^* . By the switchboard circuit construction, we know that exactly one switch variable of z must be 1. Let k be the subcircuit with this active switch: that is, $z[s_k] = 1$. Let the variables corresponding to $\mathcal{C}_k$ be $(1, x_k, \omega)$. By the input consistency constraints, we have that $x_k = x$. Thus, ω is the desired witness for $\mathcal{C}_k$.

The switches of all subcircuits $j \neq k$ is set to 0. By the input consistency constraints, this in turns implies that the input variables corresponding to it must be set to 0. Thus, the witness variables can be set to 0 as well, leading all constraints in the subcircuit to be trivially satisfied.

Sparsity: the number of non-zero elements in w^* is at most $1 + |x| + |\omega|$ and they all correspond to the one active subcircuit. $\square$

C.2. Proof of Theorem 11

Proof(sketch). Suppose prover $\mathcal{P}^*$ produces an n -step Nova proof of the statement (n, x_0, x_n) with C^* as the step circuit. From the knowledge soundness of Nova, we can extract $x_1, \dots, x_{n-1}$ and corresponding witnesses satisfying circuit C^* for each of the n steps. From Lemma 10, we know that extracting a witness satisfying C^* is equivalent to extracting a witness satisfying a particular subcircuit. This implies that we can further parse these witnesses as $\omega_0, \dots, \omega_n$ such that $F_{\text{pc}_i}(x_i, \omega_i) = x_{i+1}$ for $i \in 0, \dots, n-1$, where $\text{pc}_i = \varphi(x_i, \omega_i)$ is implied by φ being embedded into each subcircuit.

We can extend this to perform a sequence of $\beta \geq 1$ functions per step of Nova using standard circuit repetition techniques: the step circuit will be β copies of C^* with $(\beta-1) \cdot |x|$ extra constraints to enforce sequential input-output consistency between them.

What remains to be shown is that using C^* with Nova satisfies the “pay-per-use” efficiency criteria of non-uniform IVC. Recall that Nova folds two relaxed RICS instances satisfying the following equations: $Az_1 \circ Bz_1 = u_1 Cz_1 + E_1$, and $Az_2 \circ Bz_2 = u_2 Cz_2 + E_2$. Recall that in Nova’s setting of IVC, $E_2 = 0$ and $u_2 = 1$, so the cross-term $T \in \mathbb{F}^m$ that the prover commits is given by:

$$T = A \cdot (z_1 + z_2) \circ B \cdot (z_1 + z_2) - u_1 \cdot C \cdot (z_1 + z_2) - E_1$$

As a quick sanity check, suppose that $z_2 = 0$, then $T = Az_1 \circ Bz_1 - u_1 Cz_1 - E_1 = 0$. So if the incoming instance’s witness is all zeros, the error term commitment is commitment to a vector of zeros, which takes constant time to compute.

Now, consider the case where z_2 has $m > 0$ non-zero entries and the rest are zeros. We can rewrite the computation of $\bar{T}$ as follows:

$$\bar{T} = \overline{(Az_1 \circ Bz_2)} + \overline{(Az_2 \circ Bz_1)} - u_1 \cdot \overline{Cz_2} - \overline{Cz_1}$$

Without loss of generality assume that the matrix dimensions of ℓ different functions are equal and that the number of rows equals the number of columns. In our context, if z_2 is m -sparse (i.e., there are m non-zero entries and in our context $m = n_{\text{active}} + 1 + \ell \cdot (1 + |x|)$), Az_2 , Bz_2 , and Cz_2 vectors will also be m -sparse. This is because in the switchboard circuit construction, each subcircuit only accesses witnesses associated with the subcircuit. The only exception is constraints associated with switch variables. However, there are at most $1 + \ell \cdot (1 + |x|)$ of these constraints, so the sparsity of Az_2, Bz_2, Cz_2 is still guaranteed.

Since Bz_2 is m -sparse, $Az_1 \circ Bz_2$ is m -sparse, so the prover can compute $\overline{(Az_1 \circ Bz_2)}$ with $O(m)$ work. Since Az_2 is m -sparse, $Az_2 \circ Bz_1$ is m -sparse, so the prover can compute $\overline{(Az_2 \circ Bz_1)}$ with $O(m)$ work. We are given Cz_2 is m -sparse, so the prover can compute $u_1 \overline{Cz_2}$ with $O(m)$ work.

We need to show that $\overline{Cz_1}$ can be computed with $O(m)$ work. The key idea here is to have the prover cache $\overline{Cz_1}$ and efficiently update it in each iteration of IVC. In more detail, at step zero, $z_1 = 0$ (the default trivial witness in Nova), so $Cz_1 = 0$ and $\overline{(Cz_1)}$ can be computed with constant work. Then at each iteration, we update $z_1 \leftarrow z_1 + r \cdot z_2$, so the prover can do the following:

$$\overline{Cz_1} \leftarrow \overline{Cz_1} + r \cdot \overline{Cz_2}$$

At initialization, $\overline{Cz_1}$ is commitment to the zero vector, and the prover knows $\overline{Cz_2}$, so the prover can update $\overline{Cz_1}$ with constant work.

Finally, we observe that we retain pay-per-use efficiency when extending to work with β copies of C^* per step of Nova, as each step-wise segment of z_2 remains sparse. (the full vector has at most $\beta \cdot m + (\beta-1) \cdot |x|$ non-zero elements). $\square$

Appendix D.

More EVM implementation details

Capturing storage. Storage is a persistent, long-term, key-value store with 256-bit keys and values. (Persistent here refers to there being only one storage associated with a given smart contract that different transactions can perform memory operations on.) Reading the value of an uninitialized key returns 0. This peculiarity, along with the fact that its addresses are sampled from a 2^{256} -sized address space, sets storage apart from the other memory types in the EVM. Conveniently, it can still be captured by the offline memory-checking techniques Nebula employs. In fact, the original Spice protocol [44] is described for general key-value stores, of which an addressable array of memory cells is a specific case. To support this, when accessing an uninitialized key k , the prover performs a special “insert” operation, adding $(k, 0)$ to the key-value store. This is effectively a write without a preceding read. As proven in Spice, the offline memory checking protocol used in Nebula provides security against “double insertions” where a prover attempts to insert a new value to an already-initialized key. Also, instead of a memory scan over the entire address space (which is prohibitively large), it suffices for the prover to only scan the initialized keys.